technische universität
dortmund

Bachelor's Thesis

Implementing a Game Engine in Embedded Real-Time Operating Systems

Maximilian Adolenko
August 24, 2026

Supervisors:
Dr.-Ing. Reinhard Georg von der Brüggen
Kay Heider, M.Sc.

TU Dortmund University
Faculty of Computer Science
Chair of Computer Science 12 (Embedded Systems)
Design Automation for Embedded Systems Group
`https://daes.cs.tu-dortmund.de`

Acknowledgements

I would like to thank Kay Heider and Nils Hölscher for their active support and advice in carrying out this thesis and for providing all of the hardware.

Contents

1 Introduction **1**

1.1 Structure of the Thesis 1

1.2 Related Work 1

2 Fundamentals **3**

2.1 Real-Time Operating Systems 3

2.1.1 What Is a Real-Time Operating System? 3

2.1.2 Tasks 3

2.1.3 Interrupts 4

2.1.4 Queues 4

2.2 What Is FreeRTOS? 4

2.3 Game Engines 4

3 Design **7**

3.1 Requirements 7

3.2 Architecture Design of the Game Engine 8

3.3 Hardware Layer 10

3.4 Game Engine Core 10

3.5 User Layer 13

4 Hardware Implementation **15**

4.1 Components Used 15

4.1.1 Microcontroller 15

4.1.2 Screen 15

4.1.3 Input 16

4.1.4 Audio 16

4.2 System Setup 16

4.3 Practical Implementation 18

5 Software Implementation **21**

5.1 Input 21

5.1.1 Connecting the Physical Buttons 21

5.1.2 ISR 21

5.1.3 Input Service . . . 23
5.2 Logic Task . . . 24
5.3 Render Task and Graphics . . . 25
5.3.1 Render Task . . . 26
5.3.2 TileMap . . . 28
5.3.3 Textures . . . 28
5.4 Timing . . . 28
5.5 Audio . . . 29
5.5.1 Tones and Sound Effects . . . 29
5.5.2 Audio Service . . . 30
5.5.3 Audio Task . . . 30
5.6 Dynamic Frame Rate Adaptation . . . 31
5.6.1 Execution Times . . . 31
5.6.2 Deadline Misses . . . 32
5.6.3 Transfer Misses . . . 32
5.6.4 Statistics Task . . . 33
5.7 Interface for Game Programming . . . 35
5.8 Game Objects . . . 36
5.8.1 Collision . . . 36
5.8.2 Physics . . . 37
5.8.3 Movement and Dirty Tile Marking . . . 37
5.9 Init Task . . . 38

6 Proof of Concept **39**
6.1 Flatter Vogel (Flappy Bird Clone) . . . 39

7 Evaluation **41**
7.1 Evaluation Setup . . . 41
7.2 Task Approach vs. Main Loop . . . 41
7.2.1 Measurement Results . . . 42
7.2.2 Analysis . . . 44
7.3 Static and Dynamic Frame Rates . . . 44
7.3.1 Measurement Results . . . 45
7.3.2 Analysis . . . 48
7.4 Optimal Tile Size . . . 48
7.4.1 Measurement Results . . . 49
7.4.2 Analysis . . . 49

8 Conclusion **51**
8.1 What Was Achieved? . . . 51

8.2 Outlook 51

List of Figures **53**

List of Algorithms **55**

List of Source Code **57**

Bibliography **59**

Declaration of Authorship **61**

1 Introduction

Real-time operating systems (RTOSs) are commonly used in critical systems such as autonomous driving because they offer deterministic and predictable timing behavior. However, game engines would also benefit from such deterministic timing behavior, since it allows, for example, a constant frame rate to be achieved and inputs to be responded to promptly.

In this bachelor's thesis, a game engine is to be developed using structures of a real-time operating system. The engine is intended to make it possible to develop different games without having to re-implement the basic structures of games, such as input polling or rendering, over and over again. In addition, a physical handheld device is to be developed in this thesis, on which games developed with the game engine can be run and played. Furthermore, this thesis sets the goal of implementing a dynamic frame rate adaptation within the game engine.

1.1 Structure of the Thesis

First, the necessary fundamentals are presented in Chapter 2. Then, Chapter 3 presents the design of the game engine's architecture. Chapters 4 and 5 describe the implementation of the architecture. To conclude the thesis, Chapter 6 presents a game developed with the game engine, and Chapter 7 evaluates the implemented game engine.

1.2 Related Work

There is not much related work on game engine development with real-time operating systems. Some games have been developed on microcontrollers using real-time operating systems. For example, the game Snake for the STM32 [22] and an LED pattern game for the ESP32, in which the player has to replicate LED patterns [15], were developed using real-time operating systems. However, these are manual implementations that do not provide any features for development in the form of a game engine.

2 Fundamentals

This chapter explains the fundamentals that were worked out for this thesis and that help in understanding it.

2.1 Real-Time Operating Systems

This section explains the core fundamentals of real-time operating systems.

2.1.1 What Is a Real-Time Operating System?

Real-time operating systems are programs that manage resources and coordinate the timing of task execution. A core property of real-time operating systems is that the coordination of tasks is predictable and deterministic, since real-time operating systems are used in real-time applications [16].
The exact functions provided by a real-time operating system differ depending on the use case. Nevertheless, all real-time operating systems have a kernel as their basic building block. The kernel consists of a scheduler, objects, and services [16].
The scheduler in a real-time operating system coordinates the timing of task execution. Different scheduling algorithms are used to determine which task is currently given processor time. These algorithms use different properties of the tasks to distribute processor time. Kernel objects are provided so that developers can use them as basic building blocks for creating real-time applications. Services act as an interface for using the functions of a real-time operating system [16].

2.1.2 Tasks

Real-time applications are usually designed with concurrency in mind. Tasks, which are independent units of execution, are used to achieve this [16].
Tasks have different properties. These include, for example, that a priority is assigned to them [16] and that tasks must complete execution by a certain point in time, a so-called deadline [3]. Depending on the specific scheduling algorithm, the scheduler uses these properties to decide which tasks may run. For example, with "preemptive priority-based scheduling", the task with the highest priority is always executed [16]. With the "Earliest Deadline First" scheduling algorithm, the task with the earliest absolute deadline

is executed [3]. Tasks can be classified into two different categories with respect to their real-time requirements. Hard deadlines mean that severe consequences can occur if a deadline is missed. With soft deadlines, there are no severe consequences if the deadline is not met, but the application may, for example, feel sluggish as a result [1]. Tasks also differ in their execution behavior. Tasks can run once or in an infinite loop. Tasks that run indefinitely must contain a blocking call so that other tasks can be executed [16]. Endless tasks can be executed periodically or aperiodically. Periodic tasks are always activated at fixed periods, whereas aperiodic tasks have no temporal regularity in their execution behavior. Aperiodic tasks with a minimum time interval between activations are called sporadic tasks [16]. One execution of a task is also called a job [3].

2.1.3 Interrupts

Interrupts are signals that are sent to the processor, for example by hardware. When the processor receives such a signal, it stops its current task and handles the interrupt by means of an interrupt service routine (ISR) [21].

2.1.4 Queues

Queues are used so that tasks can communicate with each other and synchronize data. Tasks can put messages into the queue and read them from the queue. Messages within queues are managed according to the first-in-first-out principle [16].

2.2 What Is FreeRTOS?

FreeRTOS is a real-time operating system that is used to develop real-time applications for microcontrollers and microprocessors. FreeRTOS implements the functionalities of a real-time operating system presented above in practice. The FreeRTOS scheduler uses the scheduling algorithm “fixed-priority preemptive scheduling with time slicing”. It always executes the highest-priority task that is able to run. If several tasks with the same priority are able to run, execution time is divided equally among them. Preemption, i.e. the displacement of the currently running task, and time slicing can be disabled [1].

2.3 Game Engines

Game engines are software that can be used as a foundation for developing games. Game engines separate the actual game from components that occur again and again in games. This allows these components to be reused so that they do not have to be redeveloped for every game. The components of a game engine include, for example, rendering, audio, input handling, management of game assets such as textures or sound, physics and collision

detection, animation, and logic execution. The exact separation between game engine and game differs from engine to engine, so game engines can be kept general or be tailored to specific types of games [11].

3 Design

Before the game engine can be developed using the embedded real-time operating system, it has to be designed. To this end, the requirements of the game engine are examined below and the resulting architecture is explained.

3.1 Requirements

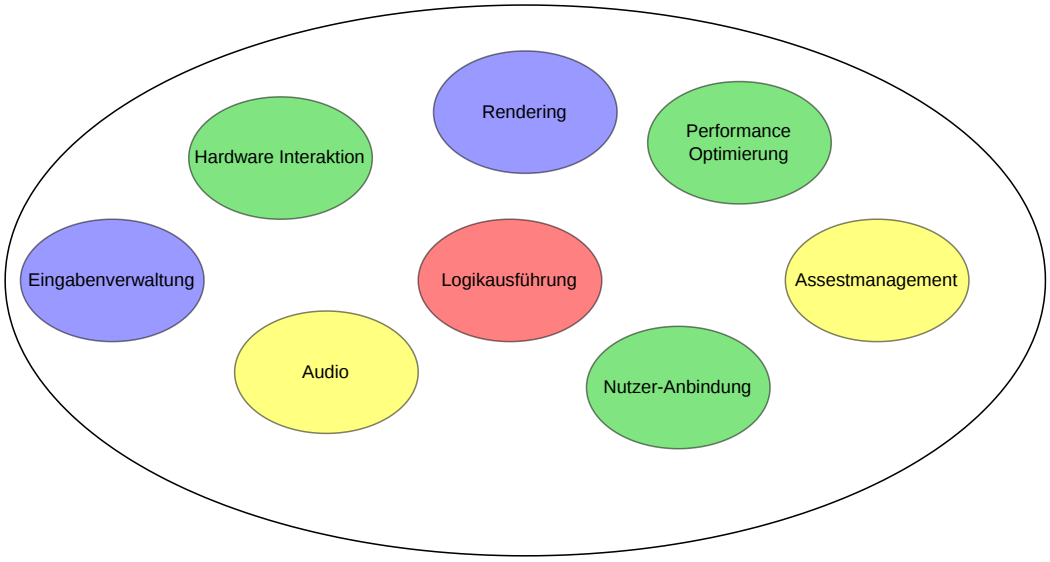


Figure 3.1: Components of a game engine

As described in Chapter 2, a game engine takes on various tasks that run in the background, such as rendering, input processing, or audio control, but also tasks that directly affect the user, such as providing collision detection and physics, the ability to program game logic, or managing assets. Figure 3.1 shows possible components of a game engine.

Therefore, the game engine implemented in embedded real-time operating systems has to meet several requirements in order to reproduce or provide the components of a game engine.
A hardware system must be available on which the game engine can be run and which the user can use as a “gaming system”.

The game engine must be able to communicate with the hardware, e.g. to send image data, read inputs, or control the audio output.
The game engine must be able to process inputs and make them available to the user as input events for use in the game logic. The communication can be handled by the real-time operating system, for example by means of interrupts.
In addition, the game engine must be able to execute the game logic and must automatically render the image from the current game state. Moreover, the two processes must run at a steady rate in order to enable a smooth image as well as steady logic execution and input processing and, ultimately, to obtain a stable frame rate.
The game engine must also update the current sound regularly and steadily to enable uninterrupted audio output. The processes could, for example, be implemented as tasks.
To be able to adapt the frame rate dynamically, the game engine must collect performance data, process it, and decide according to certain rules whether the frame rate, and thus the clock rate, is lowered or raised. Here, real-time operating systems offer suitable indicators, such as deadline misses, to check whether the system is overloaded.
The game engine must also have an interface for the user, where the user can implement the game logic or store game data such as textures or sound effects. In addition, the game engine should give the user the ability to design game scenes graphically and to manage "game objects".
The game engine should also allow the user to perform collision checks automatically, without having to implement the collision logic every time, and a few simple physics properties should be available to the user.

3.2 Architecture Design of the Game Engine

To implement the game engine in embedded real-time operating systems, the following design is suitable, which can be divided into three layers. These three layers are described and justified in more detail below. Figure 3.2 shows the entire architecture of the game engine.

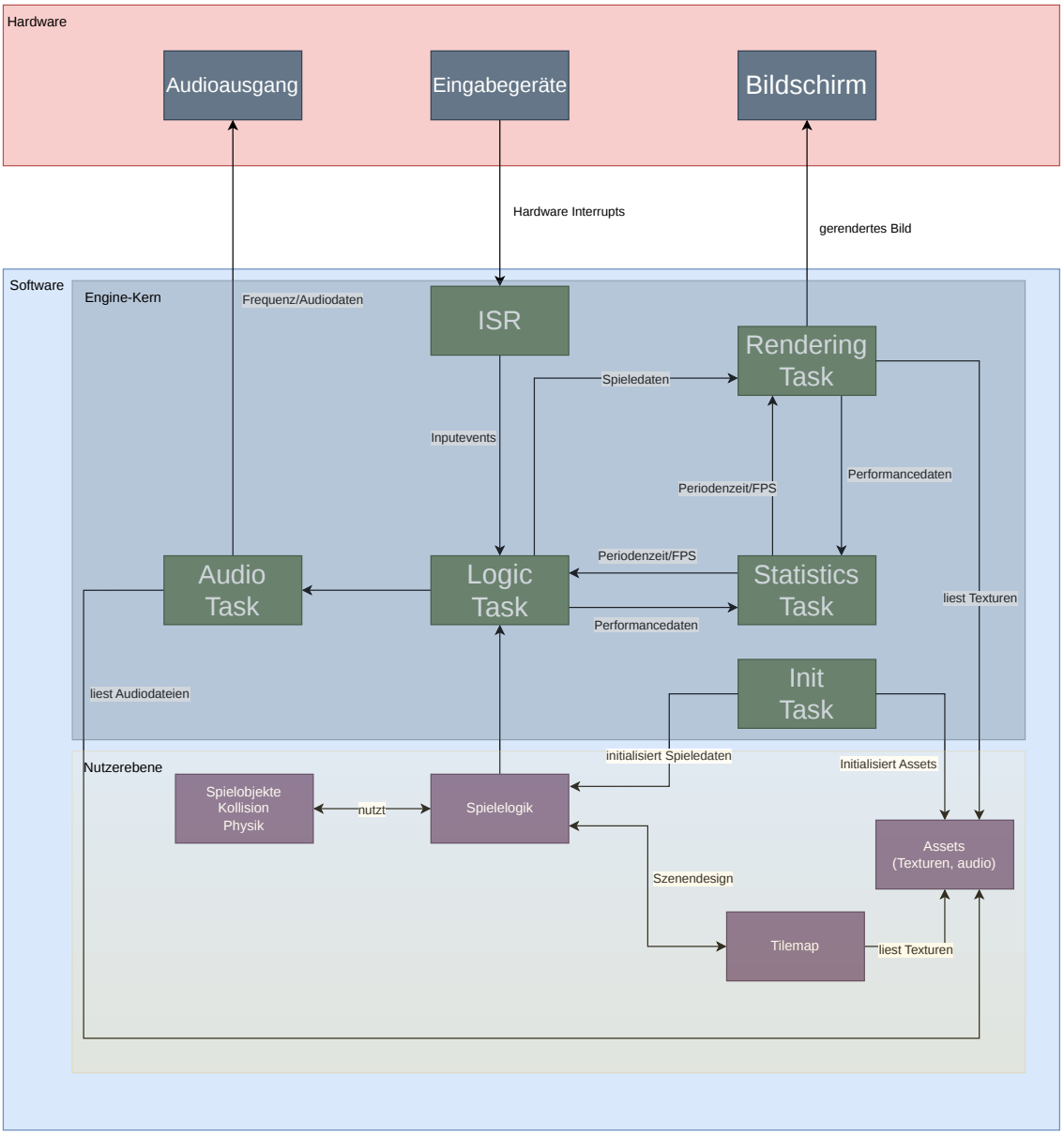


Figure 3.2: Architecture of the game engine

3.3 Hardware Layer

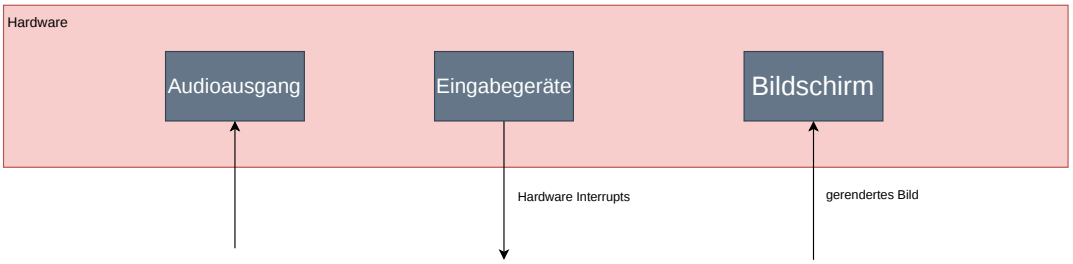


Figure 3.3: Architecture of the hardware layer

The hardware layer, shown in Figure 3.3, forms the foundation of the game engine, as it represents the embedded system on which the game engine implemented in the real-time operating system is executed. To make the hardware setup usable as a gaming system, a handheld-like design is pursued.
To display the game graphically, a screen is needed. So that the user can interact with the system and the game running on it, the hardware also needs input options. To meet the requirement that audio effects can be used, the hardware layer must also provide an audio output. A microcontroller is to be used to control the hardware components and to run the game engine. However, it is not explicitly shown in 3.3.
To keep latencies for data accesses as low as possible, e.g. when reading textures or audio files, an external storage medium such as an SD card is not used. Therefore, care must be taken during implementation to choose a microcontroller with as much internal storage as possible. Data latencies that are as low as possible are important for the game engine so that the execution times of individual tasks are not driven up by long data access times. Shorter execution times accordingly lead to higher frame rates, which enables a smoother gaming experience.

3.4 Game Engine Core

To meet the software-related requirements, the game engine is divided into two layers. The “engine core”, shown in Figure 3.4, is the part of the game engine with which the user does not have to interact directly. That is, this part of the software takes care of all tasks in the background of the game engine, which include logic execution, rendering, audio rendering, input processing, and dynamic frame rate adaptation. Concepts from real-time operating systems are particularly well suited for implementing this part of the software.

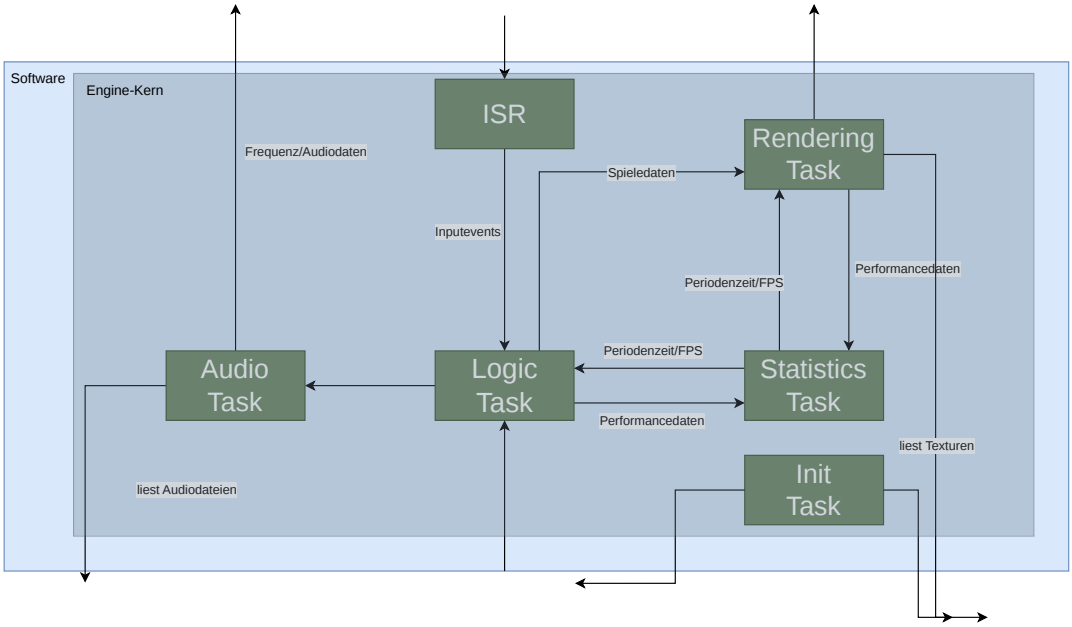


Figure 3.4: Architecture of the engine core

Inputs

Hardware interrupts, which are triggered when the input devices are actuated, lend themselves to input processing. Interrupts are particularly suitable because they allow the input to be handled as promptly as possible. Moreover, compared with polling, where the input state is actively queried, hardly any CPU time is consumed [21]. On the software side, the interrupts are to be handled by an ISR, which, upon an interrupt, translates the signal present at the input device into specific input events and then sends these to the logic task.

Tasks

The concept of tasks from real-time operating systems is suitable for implementing logic execution, audio processing, rendering, and dynamic frame rate adaptation. Tasks are suitable for these jobs for several reasons.
On the one hand, tasks allow each area of responsibility to be executed decoupled from the others, so the responsibilities of the individual jobs do not become mixed. On the other hand, tasks enable deterministic execution behavior, since they can be run with fixed period times. To be able to run tasks periodically, real-time operating systems provide a central scheduler. It therefore does not have to be developed from scratch, and one has an environment designed precisely for recurring jobs with timing requirements. This is particularly important and practical for the game engine, as it makes it possible to achieve a constant frame rate, so that games based on the game engine can run as consistently

and predictably as possible and respond promptly and reliably to inputs. In addition, deterministic execution behavior is well suited for audio processing, since a period time chosen to be as small as possible allows sound effects to be played as cleanly as possible.

The logic execution is handled by a task that is to execute the game logic implemented by the user, described in 3.5. In addition, the logic task is supplied with input events by the ISR, which the logic task then converts into the input types supported by the game engine so that the game logic can respond to them. Furthermore, after each execution, the logic task is to freeze the current game state and send it to the render task, and also to send possible audio commands to the audio task in order to control the current audio output based on game events. It makes sense to freeze the game state for rendering so that race conditions on the game data are avoided in case the logic task is already computing the next game state while the old state is still being rendered.

The render task handles image generation in the game engine. To do this, the render task consumes the game state frozen by the logic task and generates a finished frame from it, which is then sent to the screen for display. These two tasks are run together with the same period time in order to achieve the constant frame rate. The audio task ensures constant audio processing. It receives audio commands from the logic task to set or mute the current sound, and, based on the time elapsed between each execution of the task, it processes the current tone that has to be played. To obtain an audio experience that is as clean as possible, the period time of the audio task must be as small as possible.

To implement the dynamic frame rate adaptation, a statistics task is additionally created. The logic task and the render task each supply this task with performance data. Based on this performance data, the statistics task decides whether the frame rate should be raised, lowered, or left unchanged. If the frame rate changes, the new frame rate is sent to the respective tasks so that they can adjust their period times.

The init task serves only to initialize all data and has no influence on the game engine during the rest of its execution, since it is to be executed only once.

Message-based communication is used for communication between the tasks and for sending data. This type of communication has the advantage that the tasks do not have to share any data that would have to be synchronized by a mutex or semaphore. As a result, no tasks block each other during data accesses, and the tasks are thus decoupled in time. This temporal decoupling prevents the constant frame rate from being violated due to blocking.

3.5 User Layer

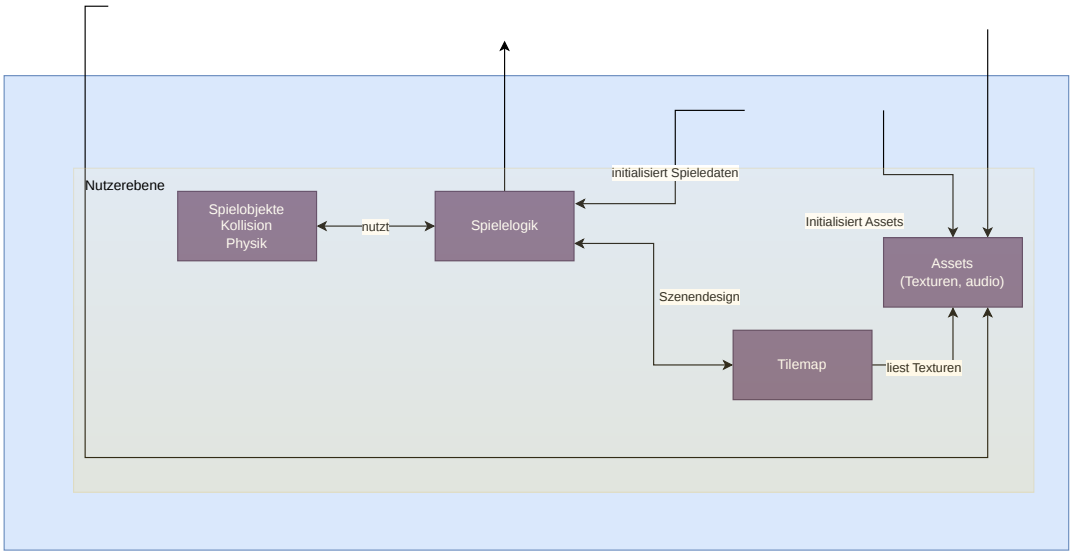


Figure 3.5: Architecture of the user layer

The user layer, shown in Figure 3.5, implements the part of the requirements in which the user has to interact directly with the game engine, e.g. to store game data or to program custom game logic.
To give the user the ability to store game data, so-called assets are implemented, which consist of two classes, “Texture” and “Sound”. These classes serve as data carriers for audio and texture data and can be programmed by the user for use in the game and placed in an “assets area” so that they are globally accessible.
So that the user can design the game or level graphically, a tile map is to be provided. It divides the screen into equally sized, square areas, where the size of the sub-areas can be chosen by the user, and a texture can then be assigned to each of these areas. Another way to design levels graphically would be to define the background or level by one large texture. However, this approach is not particularly suitable, since very large contiguous textures would have to be stored, whereas with the tile map textures can be reused to design the scene, since, for example, two tiles can be assigned the same texture. This makes it possible to save storage space. In addition, selecting the textures individually for each tile makes it easy to change the design, as only small textures have to be swapped. For the user to be able to implement custom game logic, an interface has to be available. It is also to be implemented as a class that provides a function as the means of implementation. This function is accordingly executed in the logic task to simulate the game. In addition, the interface must have access to the tile map or provide it so that the user can edit it. So that the user has access to all input events and can use them in the implementation,

the game logic interface is given access to them by the logic task via an input service. Furthermore, the game logic interface allows the user to send commands to the audio task via an audio service so that sound effects can be played.
To meet the requirement for automatic collision detection and simple physics capabilities, a “Collider” and “Physics” are provided to the user, which are also to be implemented as classes. To encapsulate and unify the use of physics, collision, and textures, “game objects” are provided as a base. They serve as a base and hold properties such as position, physics properties, and the “Collider”, and are thus logical units that can be used in the game logic implementation, preventing redundant data couplings by providing these already. They are also implemented as a class, since polymorphism can then be used to extend the base with data and functions. In addition, this unification is important for the “engine core” so that the game state can be frozen automatically and rendered afterwards, since for this, at least all textures used in the game per “object” and their positions are needed. As a result, the user accordingly does not have to implement how exactly the game state has to be encapsulated.

4 Hardware Implementation

To give the embedded game engine its hardware foundation, a handheld-like system is built in this thesis, comprising a microcontroller, a screen, input devices, and audio outputs. The following sections present the components used, describe the structure of the system, and document the implementation.

4.1 Components Used

To implement the handheld on the hardware side, the components presented below were used.

4.1.1 Microcontroller

An ESP32-P4 is used as the microcontroller, since with its 360 MHz processor [7] it offers sufficient processing power to perform more complex tasks such as rendering or game logic. The ESP32-P4 is also favored by the fact that sufficient GPIOs are available and many different interfaces are supported [7] to control the remaining components of the system. In addition, the development environment (ESP-IDF) allows simple programming of the microcontroller and its peripherals and offers an integration of FreeRTOS [6], so that the software layer of the game engine can also be implemented on the microcontroller. Furthermore, with 16 MB of flash memory [4], the ESP32-P4 offers sufficient storage space for possible textures or sound effects. The floating-point unit built into the ESP32-P4 makes it possible to work quickly with floating-point numbers [7]. This is particularly useful for calculating game states as quickly and as accurately as possible. Moreover, the low power consumption of 92 mA to 123 mA at an operating voltage of 3.3 V during dual-core data processing at a clock rate of 360 MHz [7] speaks against the need for a larger board such as a Raspberry Pi.

4.1.2 Screen

A 2.8-inch LCD display with a resolution of 240 x 320 pixels is used as the screen [24]. This allows a pleasant display of pixel graphics without demanding excessive computing effort due to an overly high resolution. For communication between the microcontroller and the display, the ILI9488 driver is available. It supports a connection via the I80

interface [24]. This enables parallel data transfer, which leads to shorter transfer times than, for example, displays connected via the Serial Peripheral Interface (SPI), and thus higher frame rates become possible. SPI only allows serial data transmission [17]. Because of the parallel data transmission, the I80 bus does require more pins than, for example, the SPI bus, but the microcontroller provides these in sufficient numbers. Other screens with even faster data transfer interfaces, such as the DSI interface, could also be considered. However, these are less common and significantly fewer microcontrollers offer a way to use this interface, which would make it harder to reproduce this work. In addition, the display supports the RGB565 format [24], in which the colors of individual pixels can be represented as a 16-bit number. This makes it possible to display up to 65536 different colors, while the memory requirement per pixel and the resulting amounts of data to be transferred remain relatively low. Thus, no external storage medium is needed for game textures either.

4.1.3 Input

Mechanical push buttons are used for input [19]. Depending on the wiring, these generate a high or low level when the button is pressed and the opposite level when it is released. This is particularly well suited for the game engine, since it is meant to process input by means of interrupts and the level change can trigger interrupts. Other input devices, such as joysticks, are not suitable for the design because they deliver values constantly and have to be read out by polling.

4.1.4 Audio

A passive piezo buzzer is used for sound output. It enables the output of different tones by changing the output frequency. Tones and melodies can thus be played without having to spend much computing effort on audio mixing. The piezo buzzer is also particularly well suited as an audio output because little memory is needed to store sound effects. Thus, no external storage medium is needed here either.

4.2 System Setup

The circuit diagram (see Fig. 4.1) shows how the individual components are put together. The entire system is powered by an external power source, which is connected to two power pins. For development, the ESP32-P4 also offers power supply via USB-C [7]. The piezo buzzer is connected to a PWM-capable pin so that the buzzer can be supplied with different frequencies [14]. In total, six mechanical buttons are used to form a “directional pad” and two “action buttons”. Technically, the push buttons provide two pins, with one pin per button connected to ground and one pin connected to a GPIO pin of the

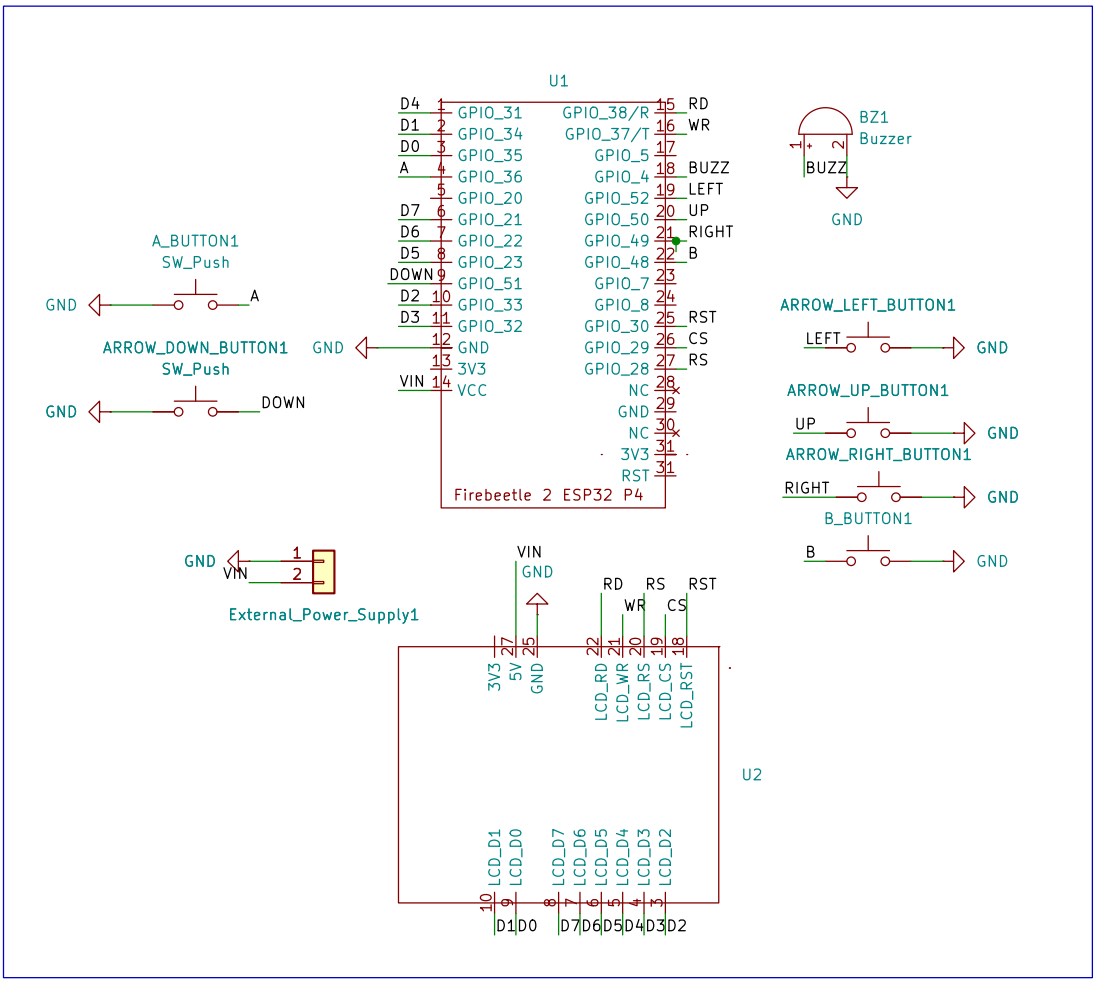


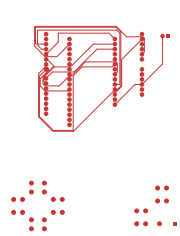
Figure 4.1: Circuit diagram of the handheld

microcontroller (see Fig. 4.1). With this wiring, the signal LOW, i.e. 0, is present at the GPIO pin when a button is pressed, and HIGH, i.e. 1, when it is not pressed. Likewise, the chosen screen is integrated into the system by connecting the eight pins of the data line and the five control pins to GPIO pins of the microcontroller. The exact connections can be read off in Figure 4.1 from the labels at each pin.

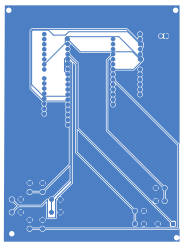
4.3 Practical Implementation

At the start of the development and implementation of the game engine, the designed circuit was first built on a breadboard in order to keep the system testable and still adjustable. The design is implemented by soldering all components onto a self-designed printed circuit board (PCB). The circuit is powered by batteries, with a voltage converter adjusting the input voltage of the circuit to 5 V.

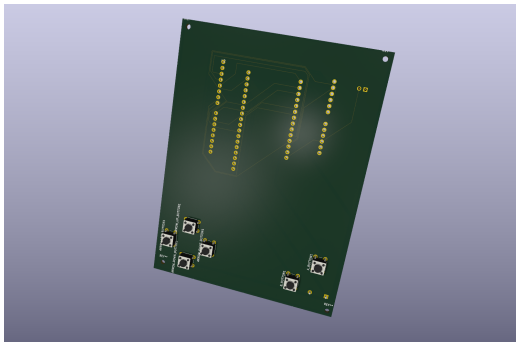
The PCB was created using the software KiCad. The PCB (see Fig. 4.2) was designed so that the push buttons were arranged on the left as a "directional pad" and on the right as "action buttons". The dimensions of the PCB are 90 mm by 120 mm. To keep the dimensions as small as possible, the screen is mounted on the front and the microcontroller on the back.
The push buttons and the piezo buzzer are soldered directly onto the board; the screen and the microcontroller are attached by soldering female pin headers onto the board and plugging the components onto them. A female pin header is likewise soldered onto the board for the power supply. Figure 4.2 shows the PCB in different views. 4.2(a) shows the PCB from the front, 4.2(b) the PCB from the back. Here it can be seen how exactly the individual components are arranged on the PCB. 4.2(c) and 4.2(d) show the PCB in a three-dimensional view from the front and the back. Some components that have already been mounted on the PCB can be seen here.



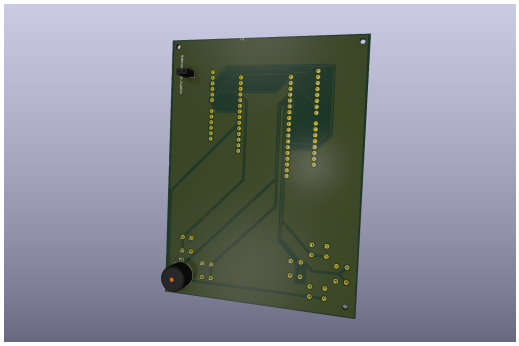
(a) Front view 2D



(b) Back view 2D



(c) Front view 3D



(d) Back view 3D

Figure 4.2: Layout and 3D view of the PCB

5 Software Implementation

The software layer of the game engine is implemented with the help of the real-time operating system FreeRTOS, since it is "integrated" directly into the development environment of the ESP32-P4. The following sections describe and explain the implementation of the software layer. The implementation of the "engine core" and the "user layer" is explained together here, since they are closely interlinked.

5.1 Input

This section explains the implementation of the input.

5.1.1 Connecting the Physical Buttons

To implement the input, the push buttons first have to be connected via the ESP-IDF interface for GPIO pins [10]. To this end, the individual GPIO pins to which the push buttons are connected are configured using the structure `gpio_config_t` [10]. Data such as the input pin or the mode (input / output) are configured here. The settings `.intr_type` and `.pull_up_en` are particularly important. Setting `.intr_type` to `GPIO_INTR_ANYEDGE` enables the push buttons to trigger an interrupt on a rising or falling edge [10], which can then be handled by an ISR. This setting is therefore essential for processing the inputs in the game engine by means of interrupts. The setting `.pull_up_en` is important because it enables a software-controlled pull-up resistor [10]. This is a resistor that pulls the GPIO pin up to a HIGH signal [2]. This is needed because a signal is present at the GPIO pin only when the button is pressed. If the button is not pressed, the pin without a pull-up resistor is a "floating input". A "floating input" means that in this case no fixed signal is defined at the pin, and thus falling or rising edges can be triggered without the button being pressed [12]. As a consequence, without a pull-up resistor, inputs could occur without actually having been made. Finally, the configuration is applied via the function `gpio_config(const gpio_config_t *pGPIOConfig)` [10].

5.1.2 ISR

So that the hardware interrupts can be processed and made available for later use in the game engine, an ISR that handles them must be available. It is advisable to develop an ISR following the "deferred interrupt processing" design. This states that an ISR should

be kept as short as possible, since an ISR always blocks and thus adds jitter to the execution times of individual tasks if the blocking times are too long, and that the processing should be done in a task instead [1]. Jitter would be very unfavorable for the design, since a stable frame rate would be at risk. For this reason, for example, game functions cannot be attached directly to the interrupt, as they can sometimes be too costly. Accordingly, the interrupt processing is implemented as follows:

```
1: Function InputHandler(void *button_type)
2: currentSignal ← signal currently present
3: if timeNow - lastTime[button_type] < threshold then
4:    return
5: end if
6: if lastState[button_type] == currentSignal then
7:    return
8: end if
9: lastState[button_type] ← currentSignal
10: lastTime[button_type] ← timeNow
11: if currentSignal == 0 then
12:    inputType ← PRESSED
13: else
14:    inputType ← RELEASED
15: end if
16: event ← inputType, button_Type, timeNow
17: send event
```

Algorithm 5.1: ISR for hardware interrupt processing

If pressing a push button changes the signal present at the pin, a hardware interrupt is triggered, which is handled by an ISR. This ISR creates an `InputEvent`. The `InputEvent` is a structure that stores the type of input (pressed or released), the exact push button, and the point in time at which it occurred. The exact input event is determined as shown in Algorithm 5.1. First, the signal currently present is read. From this, one can determine whether a falling or rising edge triggered the interrupt, since after the signal changes, the signal of the final state is present at the GPIO pin. Due to the wiring (see 4), the pin is LOW, i.e. 0, when the push button is pressed, and HIGH, i.e. 1, when the push button is not pressed. So if 0 is present at the time of the ISR, the button has been pressed; if 1 is present, the button has been released. The type of input is represented by the enumeration `INPUT_TYPE`. The button that triggered the interrupt is passed via the function signature and stored directly in the input event. To prevent mechanical bouncing, a safety mechanism is built into the ISR. Mechanical bouncing is a physical phenomenon in which the new signal, when a circuit is closed, first settles before it is stably present at the pin [20]. This means that, for example, a single button press can trigger several rising and falling edges. To prevent this, the ISR processes new interrupts at each button pin only

if they exceed a certain time span since the last accepted interrupt. While developing and testing the input, it became apparent that invalid state changes also occurred. For example, a button press was registered even though the button had been released. To prevent this, the last stable state at the button pin is stored in addition to the last time. The ISR then validates whether the state change is valid, so that only alternating presses and releases are accepted. If the interrupt is accepted, the generated `InputEvent` is sent to the logic task for processing. A queue [23] is used to send the `InputEvents`. The queue is particularly well suited for communication between the interrupt and the logic task because, for one thing, it enables message-based communication, as the input events can be placed into the queue as messages. For this reason, queues are used throughout the entire game engine as the means of communication between tasks. For another, queues allow messages or data to be placed in them safely within ISRs, since the transfer does not block [23].
This ISR is then attached via the function `gpio_isr_handler_add(gpio_num_t gpio_num, gpio_isr_t isr_handler, void *args)` [10] to each pin to which a push button is connected. As a parameter, the ISR is passed a value of the enumeration `BUTTON_TYPE` corresponding to the push button, in order to be able to distinguish between the individual buttons. In order to be able to register one ISR per pin rather than having to register a single global ISR, the function `gpio_install_isr_service(int intr_alloc_flags)` is also called [10].

5.1.3 Input Service

So that the `InputEvents` can be used by the user when implementing custom games, an input service is provided. This input service is implemented as the class `InputService`. The tasks of the `InputService` include, on the one hand, processing the raw input events into input that the user can work with and, on the other hand, managing and providing the current states and data of the individual buttons. To manage the data, the `InputService` holds six objects of the class `Button`. Since both rising and falling edges are responded to, a push button can be in three states. A push button can have been pressed, be held, or have been released. Accordingly, the class `Button` stores these three states of the respective button as well as the last times at which the button was pressed and released. From this data, the `InputService` can provide the user with the queries `wasButtonReleased(BUTTON_TYPE button)`, `wasButtonPressed(BUTTON_TYPE button)`, and `isButtonHeld(BUTTON_TYPE button)` in order to respond to the inputs in the game logic. A button counts as held from the moment it has been pressed once. This state remains until the button is released again. The stored times also make it possible to check whether a particular button has been pressed for a certain amount of time. This query is provided by `wasButtonHeldFor(BUTTON_TYPE button, uint64_t time)`. The

raw input events are processed by the function `handleInputEvent(InputEvent event)`. As in the ISR, the individual push buttons are distinguished by the enumeration `BUTTON_TYPE`. Counting, for example, how often a button was pressed is deliberately omitted. The reason for this is given in Chapter 5.2, where the way the input events are processed is described.

5.2 Logic Task

To update the game logic regularly, a logic task is implemented. For this, the task API of FreeRTOS [23] is used, since it offers the possibility of implementing periodic tasks. The logic task is implemented as shown in Figure 5.1. Before the game logic is updated, all input events arriving from the ISR (see Chap. 5.1) are first processed by passing them to the `InputService` for processing. To do this, `uxQueueMessagesWaiting()` [23] is used to determine how many input events are in the `inputQueue`, after which this number of events is read with `xQueueReceive()` [23]. The input events are processed first so that the game logic can respond to them. Here, it is deliberately not counted how often, for example, a button was pressed. The reason is that the logic task is executed once per frame, as described in Chapter 5.4. At a frame rate of, say, 30 FPS, i.e. a frame time of approx. 33.3 ms, the `InputEvents` would thus be processed every 33.3 ms. In such a time span, it would not be possible to press a button several times. Accordingly, with the implemented design, no input should be missed. Next, all drawn areas of the screen are reset. This is necessary because the render task redraws only areas of the image that have changed. More details on this are given in Chapter 5.3. So that the game logic can be simulated independently of the frame time and, for example, movements always proceed evenly regardless of the frame rate, the time elapsed between the start of the last job and the current job is also measured and passed to the game logic when it is updated. After the game logic has been updated, some input states have to be reset via the `InputService` function `resetInputAfterFrame()`. The reason for this is that the inputs "button X was pressed" or "button X was released" are valid for only one job of the task and then have to be reset so that, for example, a button press is not responded to multiple times.
In addition, after the game logic has been updated, the current game state is "frozen" so

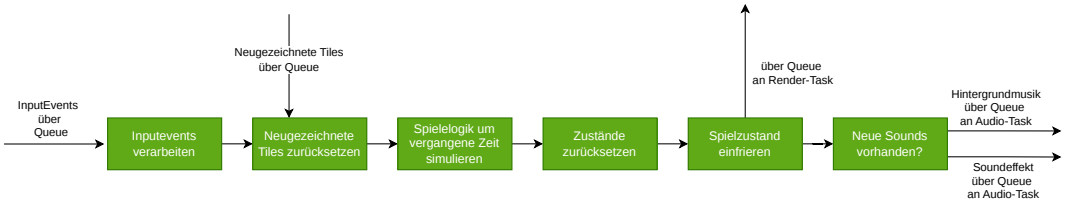


Figure 5.1: Flow of the logic task

that the render task can draw it on the screen. The game state is stored in the structure `WorldSnapshot`. Its exact contents are explained in Chapter 5.3 and its exact creation in Chapter 5.7. After the game state has been frozen, it is sent via the queue `renderQueue`. This is a queue with room for exactly one game state. This queue size, in combination with the function `xQueueOverwrite()` [23], which is designed for queues of length 1 and always overwrites their contents, ensures that the most recent game state is always available to the render task for rendering. Compared with a larger queue, game states cannot pile up if the render task cannot keep up with the clock rate, and thus no state from the past is drawn even though the actual game state has already advanced further. If the game logic triggers the command to start a sound effect or background music, these commands are sent to the audio task via two queues. After the commands have been sent, the `AudioService`, which provides the audio management, is reset so that the same audio commands are not sent multiple times. To ensure the task is executed periodically, the function `vTaskDelayUntil()` [23] of the task API is called at the end. This function blocks the execution of the logic task for the specified time span relative to the last start time of the job. As the time span, the frame rate is converted into the period time using the formula 1000[ms]/FPS, and this is converted into the unit accepted by the function with the help of the function `pd_MS_To_Tick()`. As a result, the game logic is executed evenly once per frame so that the stable frame rate is achieved (see Chap. 5.4).

5.3 Render Task and Graphics

For the game to be displayed graphically and for the user to be able to design it graphically, the game engine must provide several components. These are explained below.

5.3.1 Render Task

```
Function renderTask(void *param)
Initialize time variables
while true do
    read the most recent snapshot
    for tile in snapshot do
        take a free buffer
        mark the buffer as in use
        copy texture of the tile into the buffer
        for object in snapshot do
            if object in or touching the tile then
                copy texture of the object into the buffer
            end if
        end for
        send the buffer to the screen at the position of the tile
        wait until the next execution time
    end for
end while
```

Algorithm 5.2: Flow of the render task

```
1  bool onTransfer(esp_lcd_panel_io_handle_t panel_io,
       esp_lcd_panel_io_event_data_t *edata, void *user_ctx){
2      bool tmpForTok = false;
3      RedrawedTile t = {};
4      xQueueReceiveFromISR(transferTokens, &tmpForTok, 0);
5
6      uint16_t* buf;
7
8      if(xQueueReceiveFromISR(bufferInUseQueue, &buf, 0) == pdTRUE){
9          xQueueSendFromISR(freeBufferQueue, &buf, 0);
10     }
11     if(xQueueReceiveFromISR(tileInTransferQueue, &t, 0) == pdTRUE){
12         xQueueSendFromISR(redrawedTilesQueue, &t, 0);
13     }
14     return false;
15 }
```

Listing 5.1: Callback after a transfer

The render task serves to generate the image from the current game state. There are different approaches to generating the image. For example, in each job of the render task the entire image can be generated and sent to the screen, or only sub-areas that need to be redrawn can be generated. Since the screen's data transfer rate is limited and can thus be a bottleneck for the frame rate, this thesis follows a rendering algorithm based

on the "dirty rectangle" principle. Here, only sub-areas of the image where the game state has changed are regenerated [8]. This minimizes the amount of data that has to be sent to the screen, and a constant and as high a frame rate as possible can be ensured. The "dirty rectangle" principle does add management overhead, but it remains relatively small, since the management can be integrated into the tile map explained in the design. To implement the render task in practice, Algorithm 5.2 is implemented in a task: At the beginning of each render task job, the most recent available "snapshot", generated by the logic task, is read. To do this, the value is read from the `renderQueue` using the function `xQueueReceive()`.

The snapshot read contains all tiles of the tile map (see Chap. 5.3.2) and all game objects (see Chap. 5.8) in the game. The render task now iterates over each `Tile`. If the `Tile` has to be redrawn, a free buffer is taken and the texture of the tile is copied into it. The buffer is an array of the same size as a `Tile`. The entire image section contained in the respective `Tile` is generated in it. If there is no free buffer, the `Tile` currently being iterated is skipped and rendered in the next job. To manage the free buffers, a queue is provided that stores the addresses of the buffer arrays. Accordingly, a free buffer is read from the `freeBufferQueue` with `xQueueReceive()`. The buffer used to render the tile is also marked directly as "in use". To do this, the address of the buffer in use is placed into the `bufferInUseQueue`. This is necessary so that a buffer that is currently being sent to the screen is not simultaneously written to again. The queue thus protects a buffer from race conditions, so that no image shows two different states at the same time. In total, ten buffers are provided.

After the texture of the tile has been copied into the buffer, every game object is iterated over. If an object is located within the `Tile` currently being rendered, its texture is copied into the buffer. After all objects have been checked, the generated buffer is sent to the screen via `esp_lcd_panel_draw_bitmap()` [13]. To signal to the logic task that a tile has been redrawn and its state can thus be reset to "drawn", a `Tile` is noted as "being drawn" by placing it into the `tileInTransferQueue`. This queue is used later to release the `Tile` completely.

The logic task must be informed about the drawn tiles so that their state can be reset and they are then no longer redrawn by the rendering algorithm when this is not necessary. In principle, the logic task could reset all tiles itself, but this could lead to errors in the overall image if a tile was not rendered by the rendering, for example because no free buffer was available. If the logic task reset all tiles itself, the information that the tile still has to be redrawn would thus be lost. The LCD API of the ESP32 offers the option of registering a callback function that is executed when a transfer to the screen has finished. Registration is done via the function `esp_lcd_panel_io_register_event_callbacks()` [13]. Such a callback function is implemented to release a buffer again once the transfer to the screen has completed. To do this, the address of the buffer is placed back into the

`freeBuffer` queue (see 5.1). In addition, the `redrawedTile` queue is used here to tell the logic task that a tile has been redrawn. To ensure the task is executed periodically, the function `vTaskDelayUntil()` [23] of the task API is called at the end.

5.3.2 TileMap

As described in Chapter 3, a tile map is to be implemented that enables the user to design the game scene or background. For this purpose, the class `TileMap` is implemented. It divides the screen into a grid of equally sized areas, which can be managed via the `TileMap`. To bundle the data for each individual sub-area, the class `Tile` is created. It holds data such as the texture, the size of the sub-area, and the position in the grid. This information allows the `Tile` to be displayed graphically, and the size and position in the grid allow the absolute position to be calculated in order to check whether anything changes within this sub-area. To record that a `Tile` has to be redrawn, a flag is provided. The `TileMap` also makes it possible to assign a texture to each individual `Tile`, and by checking positions it allows the redraw flag to be triggered in order to signal that a `Tile` has to be redrawn. Since the `TileMap` is used to design the background of the game, it is drawn as the bottom layer in the rendering.

5.3.3 Textures

To manage textures, the class `Texture` is provided. It holds the width, the height, and the pixel data of the texture. The pixel data is represented as a row-major array of 16-bit integers, so that each individual pixel of a texture can be represented by a single number. The class `Texture` also holds a bit mask for the texture. This bit mask makes it possible to render only selected pixels of the texture. This allows, for example, a circle to be rendered without a colored, rectangular border. The bit mask is also provided as a row-major array that stores 16-bit values. To draw a pixel, the value 0xFFFF is used; otherwise 0x0000. Other values lead to undefined behavior. If no bit mask is specified, all pixels are considered in the rendering. If the `Texture` class is not given a valid array of pixel data, the texture is not drawn.Passing an incorrect width and height also leads to undefined behavior.

5.4 Timing

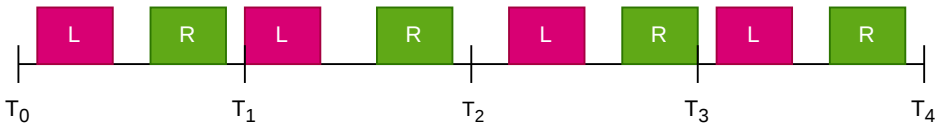


Figure 5.2: Optimal timing

For each frame during the course of a game, the logic has to be updated and the most recent image has to be rendered, as shown in Figure 5.2. To achieve a constant frame rate and thus a smooth gaming experience, the logic task and the render task must therefore be executed once per frame. To achieve this, the period time available per frame is calculated from the target frame rate, and both tasks are executed with this period. The target frame rate can be set by the user via a config file. The period time is calculated using the relation T = 1000/FPS. It is important here that the period time is given in milliseconds. The FreeRTOS function `vTaskDelayUntil()` [23] receives the period time as a number of ticks. The function `pdMS_TO_TICKS()` is used for the conversion.
The FreeRTOS scheduler always executes the task in the ready state that has the highest priority [1]. To clock the flow within a period so that the logic task is executed before the render task, the logic task is given a higher priority than the render task. Should irregularities in the execution of the tasks occur, for example if the period time cannot be met, the higher priority of the logic task ensures that the game logic stays as up to date as possible. The course of the game thus remains steady, and only the image would stutter more. In the implementation of the engine, the logic task runs with priority 4 and the render task with priority 3.

5.5 Audio

To meet the requirements for audio support in the game engine, different components are implemented in the layers of the game engine.

5.5.1 Tones and Sound Effects

To be able to store and manage sound effects, the structure `Tone` and the class `Sound` are provided. The structure `Tone` is a basic structure that stores the pitch of a tone and its duration. A tone is stored in exactly this form because a piezo buzzer (see 4.1) is used to play audio. It can only generate tones based on a frequency. The class `Sound` primarily stores an array of objects of the structure `Tone`. This stores a scale that represents a sound effect. In addition to the sound effect itself, the class `Sound` manages the playback of the sound effect by storing the current time within the sound effect, updating this time, and returning the current tone. This is used by the audio task in the game engine core to play the appropriate tone. The class `Sound` also allows setting whether a sound is played through only once or in a loop. Playing in a loop makes it possible, for example, to create background music using short scales. The class `Sound` also stores the priority of the sound effect. This is used to play only the sound effect with the highest priority if situations arise in the game logic in which several sounds could be played.

5.5.2 Audio Service

So that the user can trigger the playback of sound effects when implementing games, an audio service is provided. The audio service is implemented via the class `AudioService`. The audio service allows the user to have sound effects played on two "audio tracks". The first "track" can be used to set and play background music, and the second is used for sound effects that occur. The sounds to be played can be set via the functions `setSoundeffect(Sound* newEffect)` and `setBackgroundMusic(Sound* newBackgroundMusic)`. When sounds are set, the priority of the sounds is always taken into account, so that only the sound with the highest priority is accepted. The audio service also makes it possible to stop the sounds that are currently playing. As mentioned in Chapter 5.2, after each execution of the game logic the sounds that have been set are passed from the logic task to the audio task. To do this, they are read from the audio service, if sounds are present, and sent to the audio task via two queues. After sending, the sounds set in the audio service are reset, so that setting them again can proceed cleanly in the next logic task job.

5.5.3 Audio Task

So that the sound effects to be played can be played cleanly, a periodic audio task is implemented. It is responsible for switching the sounds to be played when necessary, updating the current sounds at small time intervals and thereby playing them, and sending the current tone that has to be played to the audio output.
To this end, the audio task is implemented as shown in Figure 5.3. At the start, it is checked whether a new sound effect or new background music is to be played. For this purpose, the queue `soundEffectQueue` is read for the sound effect and the queue `backgroundMusicQueue` for the background music. If there is a sound effect, the current sound is overwritten only if the new sound has a higher priority. This ensures that the user can determine which sounds are to be played with priority. For the background music, however, the priority is not taken into account. The reason is that it is changed less often and can always be changed whenever the user wants. When new sounds are set, their playback is started so that they play from the beginning again. If no new sounds were set but there are current sounds, they are updated by the elapsed time. When the sound effect or the background music is over, the current sounds are set to empty. After the update, the tone that is sent to the audio output is determined. The sound effect is always played with higher priority than the background music. That is, if there is a current sound effect, its current tone is played. If there is no sound effect but there is background music, its current tone is sent; otherwise no tone is played at all.
To play the current tone, the frequency of the tone is generated at the pin to which the piezo buzzer is connected. The functions `ledc_set_duty()`, `ledc_update_duty()`, and `ledc_set_freq()` are used for this [14]. To update sound effects as accurately as possible,

the audio task is executed independently of the frame rate with a period time that is as small as possible. This can be set via a config file and is set to 5 ms by default. The priority of the audio task is set to 5 so that it is always executed immediately.

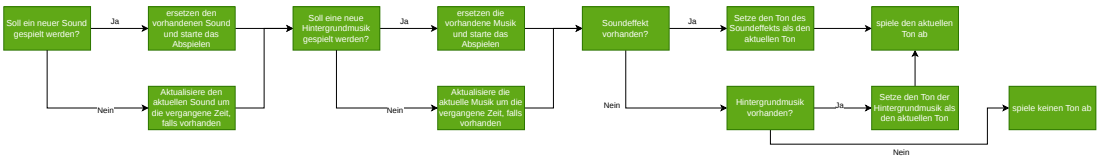


Figure 5.3: Flow of the audio task

5.6 Dynamic Frame Rate Adaptation

To meet the requirement for dynamic frame rate adaptation, some performance data must be collected while the game is running in order to calculate a resulting frame rate from it and to adapt the current frame rate accordingly. In this thesis, the execution times of the logic task and render task, the deadline misses, and the transfer misses, i.e. whether the entire image arrived at the display before the next render task job, are measured in order to determine the target frame rate from them. The target frame rate is determined by the statistics task.

5.6.1 Execution Times

pictures/AusfuÌÍhrungszeiten.pdf

Figure 5.4: Period time is too large

As shown in Chapter 5.4, one execution of the logic task and one execution of the render task are performed per frame. This means that the execution time of the logic task added to the execution time of the render task should equal the period time of the current frame rate. If a frame rate is set statically, the execution times added together may be too large. This would mean that the statically set frame rate is too high. If the added execution times are significantly smaller than the period time of the set frame rate, as shown in Figure 5.4, the frame rate can be set even higher, since the period time is not fully utilized. To be able to determine the utilization of the period time, the execution times are measured for the logic task and the render task. For this purpose, a large array of 10000 values

that stores start-stop time pairs is created for each task. The time is measured by the function `esp_cpu_get_cycle_count()` [18]. It merely reads the CPU cycle counter from a register, which entails little overhead.

5.6.2 Deadline Misses

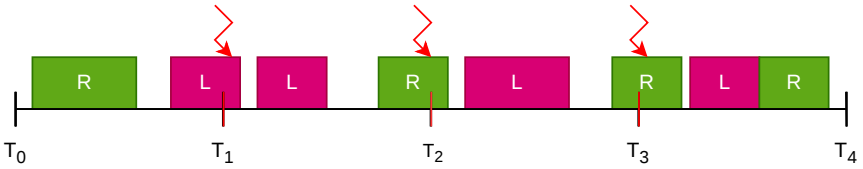


Figure 5.5: Deadline misses

Deadline misses occur when a periodic task finishes after its deadline, as shown in Figure 5.5. In the context of the game engine, a deadline miss means that a logic task or render task job finishes only in a later frame. If deadline misses occur, this means that the system cannot in practice keep to the period time of the set frame rate. To be able to react to this, every deadline miss of the logic task and render task is measured. For this purpose, the function `xTaskDelayUntil()` provided by the FreeRTOS API is used for the periodic waiting instead of the function `vTaskDelayUntil()`. The function `xTaskDelayUntil()` has the advantage that it returns `pdFALSE` if the next start time of a task's period is in the past [23]. Accordingly, this means that the task has missed its deadline. If a deadline miss occurs in the logic task or render task, they signal this to the statistics task via a queue. When a miss occurs, a token is placed in the `Deadline-MissedSignal` queue. The tokens are boolean values without any meaning. They were chosen because they consume little memory. From the number of tokens in the queue, the statistics task can then determine the number of deadline misses.

5.6.3 Transfer Misses

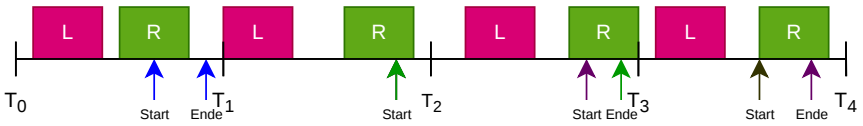


Figure 5.6: Transfer misses

A transfer miss occurs when a transfer of image data of a previous frame is still taking place during the next job of the render task. This case is shown in Figure 5.6. Accordingly, transfer misses are signals that the target frame rate is not supported by the bandwidth of the screen and that the image cannot in practice be transferred quickly enough in each

period. To be able to measure transfer misses, the render task (see 5.2) and the callback function (see 5.1) are extended. For each transfer start, a token is placed in a queue, similar to the deadline misses. The tokens are again boolean values without any meaning. The tokens in `transferTokens` are then released again by the callback (see 5.1), since it is called as soon as a transfer has finished. In addition, at the beginning of its job the render task checks whether there are still tokens in the `transferTokens` queue. If so, this is the case where data of the previous frame is still being transferred even though the next one has to be rendered. If this case occurs, the render task passes this on to the statistics task as a signal via a queue with only one element. The value that is transferred has no meaning here. The statistics task can therefore read from the contents of the queue alone whether a transfer miss occurred. So that a transfer miss is not signaled to the statistics task every time, for example, only one transfer is still ongoing, the number of tokens in the `transferTokens` queue must exceed a certain threshold. This can be set variably. No fixed value can be set because, for example, one or more ongoing transfers are weighted differently for different tile sizes. The weighting for different tile sizes arises because, in a transfer miss, areas of the image of different sizes have not yet been redrawn on the display. A transfer miss with a tile size of 10 * 10 pixels would therefore not be as critical as a transfer miss with a tile size of 80 * 80 pixels.

5.6.4 Statistics Task

```
Function statisticTask(void *param)
Initialize time variables
while true do
  if deadline misses present then
    decrease current frame rate by X
    wait until the next execution time
  end if
  if transfer miss present then
    decrease current frame rate by Y
    wait until the next execution time
  end if
  calculate the average of the execution times
  calculate possible frame rate
  if possible frame rate higher than current frame rate + W then
    slightly increase the current frame rate by at most Z
  end if
end while
```

Algorithm 5.3: Flow of the statistics task

The statistics task processes the collected performance data and, based on it, decides which frame rate the game engine should aim for. If a transfer miss is signaled to the statistics task, the current frame rate is immediately reduced by an adjustable value X and no further performance data is examined. If the number of deadline misses exceeds an adjustable threshold, the FPS is likewise reduced here by an adjustable value Y. If deadline misses occur, no further performance data is considered either. The values X and Y should ideally be set according to the developer's sense of what feels right, so that the adaptation feels as smooth as possible. If they are set too high, for example, jitter can occur when the frame rate is adjusted due to transfer or deadline misses. When the frame rate is lowered, it is limited by a minimum, which can be set. The minimum ensures that a game does not "freeze". In both cases, no further performance data is examined, since the game engine cannot in practice support the current frame rate and further performance data would add no value to the decision on the target frame rate.
If no deadline or transfer misses occur, the average execution time of a frame is calculated from the last N execution time values. The number N can also be set here. Based on experience from implementing the game engine, the number N was set depending on the period time of the statistics task. The implementation of the game engine makes it possible to set the period time so that the frame rate can be adapted as flexibly as needed. Accordingly, N should be set differently so that only the most recent measurements are analyzed in each statistics task job. During implementation, analyzing the last 20 measured execution times proved effective for a period time of 1 second. The possible frame rate is calculated from the average execution time of an entire period. An adjustable buffer is also included here, which describes what fraction of a frame the added execution time of the render task and logic task takes up. A buffer must be included because, besides the two tasks, other tasks and the real-time operating system also take up processing time. Without a buffer in the possible frame rate, the calculated possible frame rate would be too high, which could directly lead to overload.
If the possible frame rate is lower than the current frame rate, the frame rate is not lowered for the time being. The reason is that the game engine can in practice maintain this frame rate and the buffer was therefore chosen slightly too generously. If the possible frame rate is higher than the current one, the current one is raised. To avoid large jumps in the adaptation, which in practice led to image flicker during development, the frame rate is increased only in adjustable steps of size Z. During implementation, a value of 5 proved practical. Image flicker also occurred when constantly switching between two frame rates. Therefore, the switch to the higher frame rate is made only if it is higher than the current frame rate by an adjustable value W. Based on experience, no image flicker occurred when the frame rate was adjusted only if the difference between the new possible frame rate and the current frame rate was greater than 1.
So that the frame rate cannot grow toward infinity and thereby crash the game engine, the

frame rate is limited by an adjustable maximum. The priority of the statistics task is set to 6. This makes it the task with the highest priority in the game engine. The reason is that the statistics task is always executed by the scheduler when it is ready. This ensures that the statistics task is always allocated processing time, even under overload, so that it can then handle the overload.

5.7 Interface for Game Programming

So that the user can implement games independently of the engine core, an interface is provided for this purpose in the game engine. The interface is implemented by the class `World`. The main tasks of this class are to provide the functions of the engine and to give the possibility of implementing the game logic. So that the user has access to features such as the tile map (see Chap. 5.3.2) or the audio service (see Chap. 5.5.2), the class `World` provides instances of the `TileMap`, the `InputService`, and the `AudioService`.
To be able to implement the game logic, the class `World` provides the functions `init()` and `update(uint64_t deltaTime)`. The function `init()` serves to initialize the game. Here, for example, all game objects (see Chap. 5.8) can be created at the start of the game or the design of the `TileMap` can be defined. The function is called only once at the beginning of the game in order to prepare the game.
The function `update(uint64_t deltaTime)` contains the actual game logic and is called once per logic task job. In this function, it is possible, for example, to respond to inputs, trigger the playback of sounds, or check collisions. No fixed structure for the implementation within the `update(uint64_t deltaTime)` function is prescribed, so that as many different games as possible can be implemented. The parameter `uint64_t deltaTime` receives the time elapsed since the last execution of `update(uint64_t deltaTime)`, which is determined by the logic task (see Chap. 5.2). This time can then be used in the game implementation to execute the game logic independently of the period time of a frame. Chapter 6 explains the implementation of an example game in more detail in order to illustrate how to work with the game engine and how it functions. Another task of the class `World` is to freeze the current game state after an update. For this purpose, the function `createSnapShot()` is provided. It copies all tiles of the tile map and all game objects from the object register into the structure `SnapShot`, which is then sent from the logic task to the render task.
The object register serves as the management point for all game objects contained in the game. It is implemented via the class `ObjectRegister`. The register is used above all so that all game objects registered there are taken into account in the `SnapShot`. Because the user can register the game objects used there, the user no longer has to manually implement the creation of the `SnapShot` depending on the contents of the game. In addition, the `ObjectRegister` can be used for further functionalities, which are explained later.

5.8 Game Objects

In the game engine, game objects serve as a basic unit that bundles certain data and is used as a technical interface within the engine. Game objects are implemented by the class `GameObject`. The class holds data such as the position of the object within the game or the texture of the object. These two pieces of information alone are used to implement `SnapShot` generation and rendering automatically. This means that the user only has to create suitable game objects in the implementation in order to have all the content they want to display rendered automatically. The game objects do not even have to execute any particular logic and can be used purely for display. Nevertheless, game objects lend themselves to providing certain features for the game engine. The following features were provided as part of this thesis.

5.8.1 Collision

Since game objects already store a position, they are well suited to providing automatic collision detection. Automatic is meant here in the sense that the user does not have to implement the checking logic themselves, e.g. whether two instances in a game collide with each other. The collision logic is provided via the class `Collider`, which is held as an instance by the class `GameObject`. The class `Collider` in turn serves as an abstract superclass to provide a general interface for collision. This allows collision detection to be provided for different shapes by creating subclasses of the class `Collider`. In the game engine, collision detection is provided for rectangles and circles through the subclasses `BoxCollider` and `CircleCollider`. A game object can thus be assigned a hitbox in the form of a circle or a rectangle. For collision checking, the functions `touchesAndOverlapsWith(Collider& otherCollider)` and `overlapsWith(Collider& otherCollider)` are provided. The `Collider` of another object can be passed to them, and a Boolean value is returned that indicates whether a collision takes place. The two functions differ in that the latter only checks whether two hitboxes overlap, while the former additionally checks whether two hitboxes touch. In addition, collision types are managed via the class `Collider`. These collision types are used, on the one hand, to provide a more general collision query. On the other hand, they are used to implement a form of movement that is stopped when colliding with a noted collision type. The collision types are managed via the enumeration `COLLISION_TYPE`, which the user can extend as desired. A `Collider` can hold its own type and the types with which the game object is to collide during movement.
Because all game objects have to be registered in the object register, the class `World` can, in combination with the collision types, offer to check whether an object collides with a certain collision type. To do this, the register is simply iterated over and the collision with each object is checked. The function is offered via `World` here because this class holds the register, so the register and the objects themselves do not have to be linked to each other.

5.8.2 Physics

In addition, simple physics properties that can be applied to game objects are provided. They are implemented by the class `Physics`. This provides that, on the one hand, a gravitational force and, on the other hand, a fixed force for a certain time can act on a game object. Both forces ultimately result in a velocity, which leads to the movement of a game object.

5.8.3 Movement and Dirty Tile Marking

To enable smooth movement over time, the class `GameObject` holds a movement direction and a speed. Further types of movement can be provided without additional information. The execution of movements itself is implemented not by the class `GameObject` but by the interface `World`. The reason for this is dirty tile rendering. If the game's image changes, the sub-areas that change must be marked as needing to be redrawn. Cases in which this happens are movements, i.e. changes of position, and texture changes. These two cases are taken into account and handled by the game engine. This is done by the interface `World`, since it provides the tile map and the game objects of a game are managed within the game logic through the object register. The classes `GameObject` and `TileMap` thus remain decoupled from each other, and no tight coupling arises. In the case of a movement, checking which sub-areas have to be redrawn is divided into two steps. Before the movement, all tiles at the old position are marked so that the object is, if applicable, no longer rendered there. After the movement, all tiles at the new position are marked in order to finally render the game object. In the case of a texture change, the tiles at the position of the object are marked once. If further cases arise when implementing a game in which tiles have to be marked, the user can trigger this manually via the function `triggerTileMapToCheckRedraw(GameObject* o)`.
In total, three types of movement are provided: game objects can be moved to a specific position, their position can be changed by a specific value, and a uniform movement based on the held speed and movement direction can be performed. For the uniform movement, it is additionally possible to perform a movement with collision checking (see Chap. 5.8.1). Here, a game object is moved only until it collides with a game object that holds a collision type that is noted as blocking for the moved game object. In addition, the uniform movement moves the object only as far as corresponds to the time elapsed since the last game update.
If one wants to extend the class `GameObject` with game-specific functionality, the concept of inheritance can be used here. For example, further data types or functions can then be added. Since the class `GameObject` is kept very thin, there are not many structural restrictions on how it can be extended. Different implementations of games with different designs are therefore possible as well.

5.9 Init Task

The init task serves to initialize the game, the textures, and the sounds. The init task was added to the design because, during implementation, situations arose in which game objects were initialized before a texture that they actually use. As a result, the game objects were not rendered. This was because game objects, textures, and sounds are managed as global objects across different files. There is no fixed initialization order here, so an object can be created before its texture. The init task fixes the order so that all assets are initialized before the game and thus before the game objects. The task runs only once, when the game engine starts.

6 Proof of Concept

As a proof of concept and in preparation for the evaluation, a game was implemented with the game engine. The following briefly explains how the game engine was used for this purpose.

6.1 Flatter Vogel (Flappy Bird Clone)

For the Flappy Bird clone, here called "Flatter Vogel" (German for "fluttering bird"), three textures were created. These are one for the bird, one for the pipes, and one for the background. For the bird texture, the bit mask feature was used so that there is no colored, rectangular border around the bird.
A total of seven game objects were created for the bird and the pipes. The game object that represents the bird in the implementation is assigned the bird texture and a `BoxCollider`. The pipes are represented by the remaining six game objects, which are grouped into pairs. Why exactly six pipes are used is explained later. The matching texture and a `BoxCollider` are likewise assigned to these. Finally, the background texture is assigned to every tile of the tile map. The implementation uses tiles of 20 by 20 pixels. The initialization and assignment take place in the function `init()`.
The actual game logic is implemented in the function `update(uint64_t deltaTime)`. In each run, it is first checked whether the "A" button has been pressed. If so, an upward force is applied to the bird for 30 ms. So that the forces of earlier jumps do not mix with and amplify the new force, they are reset. For the bird to fall downward, a downward gravitational force acts on this game object. So that it does not cancel out the jump, it too must be reset before the jump is executed. Once the input has been checked, the velocities are calculated from the acting forces, and the bird is moved accordingly. In addition, the pipes are moved at a constant speed. Exactly six pipes are used to create an endless effect. They were positioned so that at most two pairs of pipes are visible on the screen at any time. This way, the pair of pipes that is not visible can always be appended at the back. A pair is appended at the back when it leaves the left edge of the screen. In addition, in each run it is checked whether the bird collides with one of the pipes. If so, the game is stopped and can be restarted. It is also checked whether the bird has passed a pair of pipes. For this purpose, a flag `passed` is held for each pipe. To make this possible, the class `GameObject` was extended by inheritance to the class `Pipe` in order to hold the

flag. The bird has passed a pair of pipes when it is to the right of the pair. For every pair passed, the score in the game is increased. The flag serves to mark that a pair has already been passed. This prevents the score from being increased in every run just because a pair of pipes is to the left of the bird. The flag is reset when the pair is appended at the back. When the bird collides with a pipe, the score is output via the serial monitor.
Figure 6.1 shows an excerpt of the game running on the finished handheld.

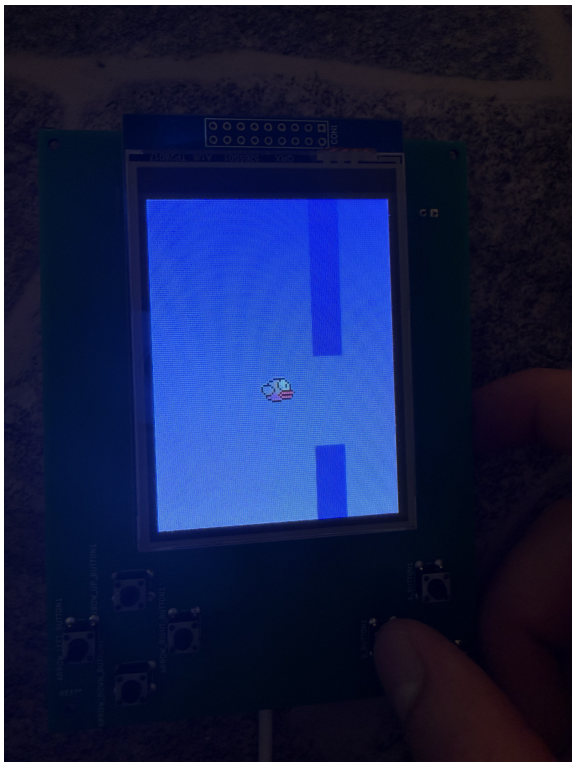


Figure 6.1: Excerpt of the game

7 Evaluation

To evaluate the game engine, several measurements were carried out. The execution of the measurements and the results are presented below.

7.1 Evaluation Setup

For the evaluation of the game engine, the hardware presented in Section 4.1 of the hardware chapter was used. The hardware components had already been mounted on the PCB. The game engine was built and flashed with ESP-IDF version v6.0. The version of the FreeRTOS kernel used was V10.5.1. During the measurements, the CPU clock rate was set to 360 MHz, and the FreeRTOS tick frequency was 1000 MHz. The function `esp_timer_get_time()` was used to measure times [5], since with a resolution of 1 microsecond it allows time measurements that are as accurate as possible.

7.2 Task Approach vs. Main Loop

In addition to the implementation of the game engine, a naive main loop approach was implemented for the evaluation. Here, the entire game was programmed manually in a main loop. This also means that the input was queried via polling and that rendering had to be implemented manually rather than being handled automatically as in the developed engine. To compare the performance and robustness of both approaches, the frame time was measured over a period of time for both approaches. To measure the frame times as accurately as possible, for the game engine the time between two render task jobs was measured, and for the main loop the time between two iterations was measured. In total, two measurements with three runs of two minutes each were carried out per approach. During the measurements, Flappy Bird was played manually. For the game engine, the dynamic frame rate adaptation was enabled. It was configured as follows:
The start FPS value was 30 FPS (`STARTFPS` = 30.0f), and the statistics task was executed with a period time of 1 second (`STATSPERIODINMS` = 1000). The minimum achievable frame rate during the measurement was 20 FPS (`MIN_FPS` = 20.0f) and the maximum 120 FPS (`MAX_FPS` = 120.0f). The statistics task always used the last 20 measured execution times (`STATSVALUESCOMPARECOUNT` = 20) to calculate the currently possible frame rate. In addition, the buffer share within the frame time was set to 20 % (`UTALIZATION_BUFFER`

= 0.2f). The frame rate was changed if it already differed from the old one by the value 1 (`FPS_THRESHOLD` = 1); at most, the current frame rate was changed by the value 5 (`FPS_MAX_OFFSET_PER_ITERATION` = 5). The statistics task reacted to five deadline misses (`DEADLINE_MISS_THRESHOLD` = 5) with a frame rate reduction of two FPS (`FPS_DECREASE_ON_DEADLINE_MISS` = 2). A transfer miss was registered when five transfers (`TRANSFER_MISS_THRESHOLD` = 5) were still open when the next render job started; the reaction was a frame rate reduction of 2 FPS (`FPS_DECREASE_ON_TRANSFER_MISS` = 2).

The tile size was set to 20 * 20 pixels during the measurements.
During the first measurement, the performance of both approaches was measured without additional load in order to obtain a basis for comparison. In the second measurement, a task that generates background load was added to both the game engine and the naive loop approach. For this purpose, a task with priority 1 was created that generates a random number N between 10000 and 100000 and then performs a floating-point calculation N times. The task was executed with a period time of 50 ms. The task for generating the background load could also be used in the main loop implementation, since within ESP-IDF the main loop is started by the main task and is accordingly also executed by it [9]. Note that the implementations of the game engine and the naive main loop do not have the same features. The game engine supports, for example, audio and the dynamic frame rate adaptation, while the main loop does not. In addition, the game engine uses tile rendering, whereas the main loop renders the entire screen.

7.2.1 Measurement Results

The following measurement results were obtained. In this thesis, only one run is shown per measurement, since the runs behaved similarly.
Figure 7.1 shows a measurement run without background load. The x-axis represents the elapsed measurement time in milliseconds, and the y-axis the frame times measured over the measurement time in milliseconds. The blue graph shows the frame times of the game engine implementation, and the red graph represents the frame times of the main loop implementation. From the frame times, it can be seen that the main loop approach achieved a lower frame time than the game engine, and thus a higher frame rate. It can also be seen that the game engine implementation showed many peaks.
Figure 7.2 shows a measurement run with background load. Here too, the x-axis reflects the elapsed measurement time in milliseconds, and the y-axis the frame times measured over the measurement time in milliseconds. The blue graph shows the frame times of the game engine implementation, and the red graph represents the frame times of the main

loop implementation. Here it can be seen that the game engine implementation achieved a lower frame time than the main loop implementation. The peaks in the frame times of the main loop implementation also show that the frame times fluctuated considerably.

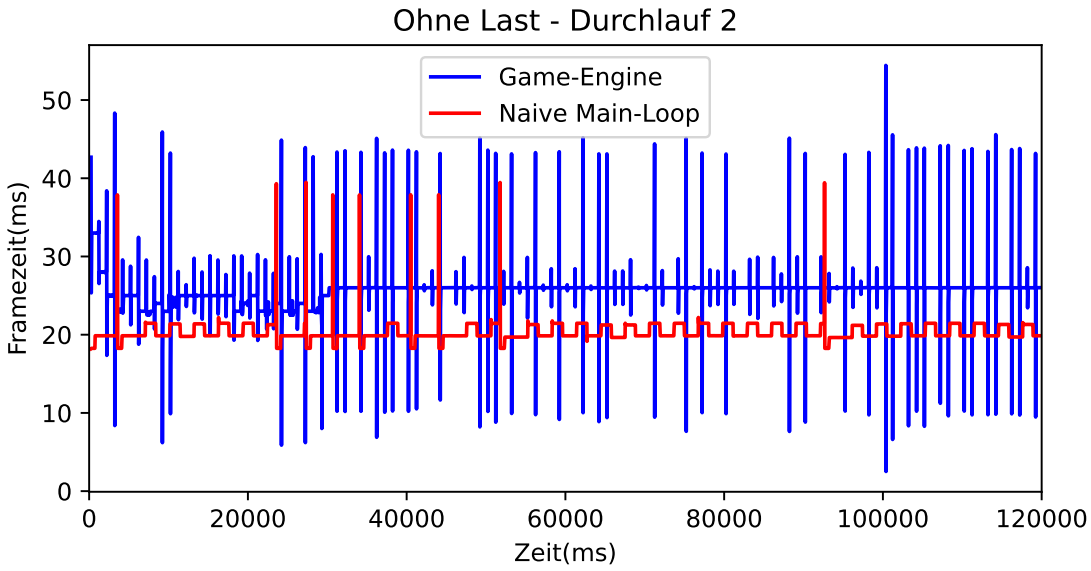


Figure 7.1: Run without load

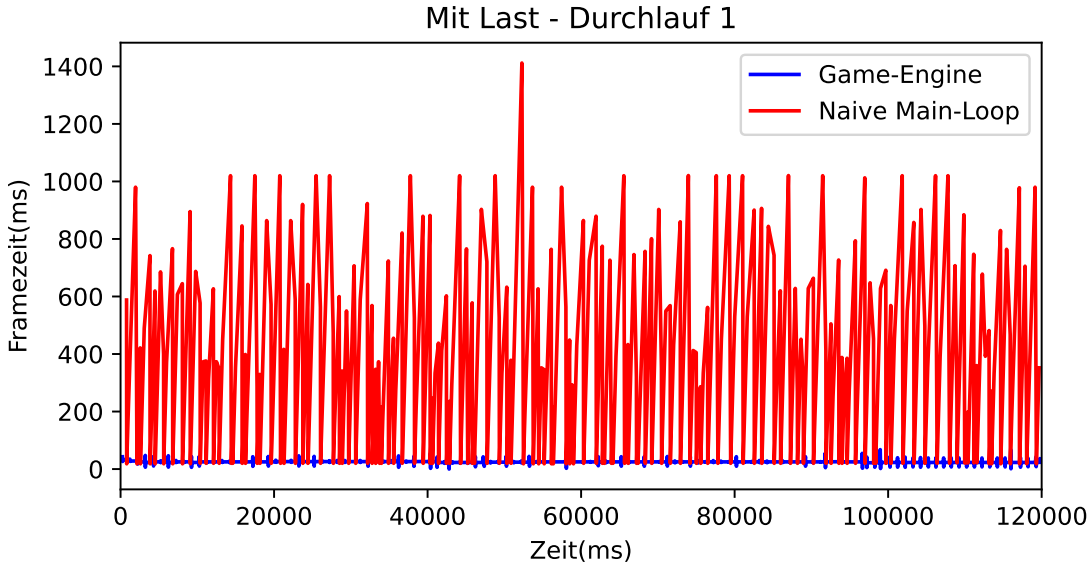


Figure 7.2: Run with load

For the measurement without load, the game engine has an average frame time of 25.32 ms and the main loop 19.86 ms. The standard deviation of the game engine is 3.15, that of the main loop 1.08.

For the measurement with background load, the game engine achieved an average frame time of 25.13 ms and the main loop 304.18 ms. The standard deviation of the game engine is 3.78, that of the main loop 332.03.

7.2.2 Analysis

The measurement without load shows that the naive main loop approach performs somewhat better than the game engine approach, since the loop achieves a frame rate of approx. 50 FPS and the game engine only approx. 39 FPS. However, considering that, in the settings of the dynamic frame rate adaptation, 20 % of the summed execution times is included in the frame rate as a buffer, the game engine would also achieve a frame rate of approx. 50 FPS without this buffer. Particularly striking are the large peaks in the frame times of the game engine. These probably arise because the load can differ from frame to frame due to tile rendering, because unfavorable interrupts from inputs occur, or because the dynamic frame rate adaptation forces deadline misses, so that rendering is preempted for a short time. The main loop approach runs so stably because the load per frame remains identical. Looking at the measurement with the generated background load, one sees the clear advantage of the RTOS approach for the game engine over the naive main loop in terms of robustness. The performance of the game engine hardly changes compared with the measurement without background load. The reason is that the FreeRTOS scheduler allocates processing time to the generated task only when there is spare capacity and, because of the higher priorities of the game engine tasks, gives the entire game engine precedence in execution. The main loop approach lacks this "protective structure", which causes the frame rate to collapse completely. With an average frame time of 304.18 ms, a frame rate of approx. 3 FPS is reached. With such a frame rate, the game was not playable at all.

7.3 Static and Dynamic Frame Rates

To assess how the dynamic frame rate adaptation implemented in this thesis affects the gaming experience, the frame time behavior is measured at different start FPS values for static versus dynamic frame rates. For this purpose, five different start FPS values are measured, with two runs of two minutes each. To measure the behavior at static FPS, the dynamic frame rate adaptation is disabled. Measurements with the dynamic frame rate adaptation are carried out with the following settings: The start FPS value was 30 FPS (`STARTFPS` = 30.0f), and the statistics task was executed with a period time of 1 second (`STATSPERIODINMS` = 1000). The minimum achievable frame rate during the measurement was 20 FPS (`MIN_FPS` = 20.0f) and the maximum 120 FPS (`MAX_FPS` = 120.0f). The statistics task always used the last 20 measured execution times (`STATSVALUESCOMPARECOUNT` = 20) to calculate the currently possible frame rate. In addition, the buffer share within the

frame time was set to 20 % (`UTALIZATION_BUFFER` = 0.2f). The frame rate was changed if it already differed from the old one by the value 1 (`FPS_THRESHOLD` = 1); at most, the current frame rate was changed by the value 5 (`FPS_MAX_OFFSET_PER_ITERATION` = 5). The statistics task reacted to five deadline misses (`DEADLINE_MISS_THRESHOLD` = 5) with a frame rate reduction of two FPS (`FPS_DECREASE_ON_DEADLINE_MISS` = 2). A transfer miss was registered when five transfers (`TRANSFER_MISS_THRESHOLD` = 5) were still open when the next render job started; the reaction was a frame rate reduction of 2 FPS (`FPS_DECREASE_ON_TRANSFER_MISS` = 2). The tile size was set to 20 * 20 pixels during the measurements. For each start FPS value, the config flag `STARTFPS` was set accordingly.

7.3.1 Measurement Results

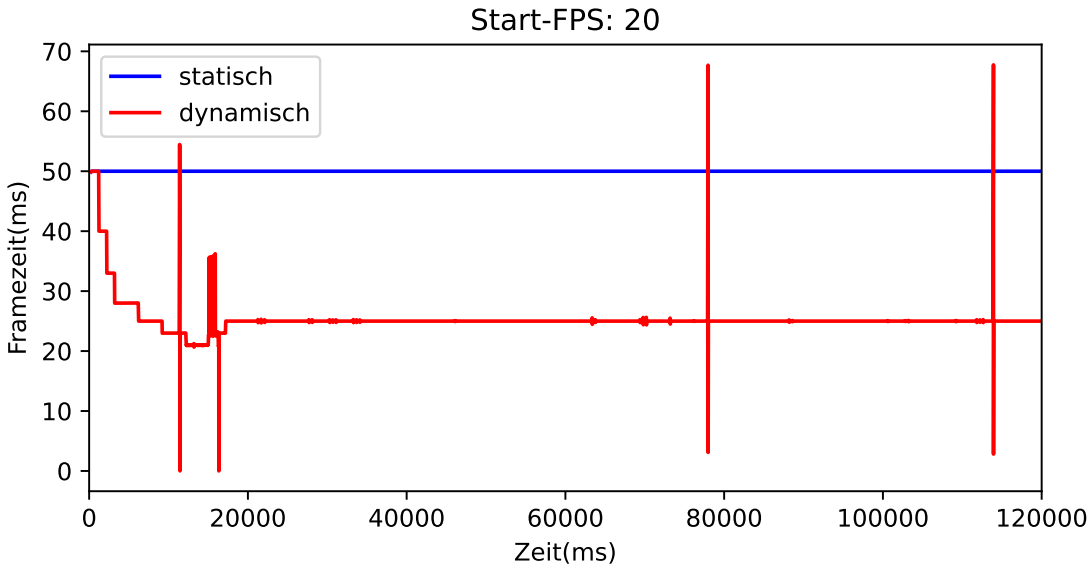


Figure 7.3: Run with 20 FPS

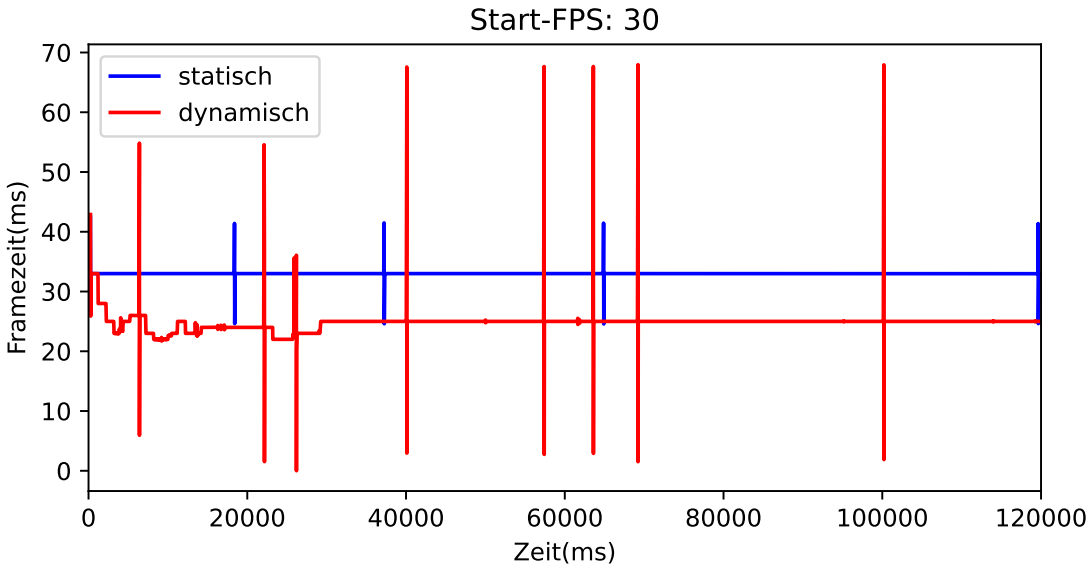


Figure 7.4: Run with 30 FPS

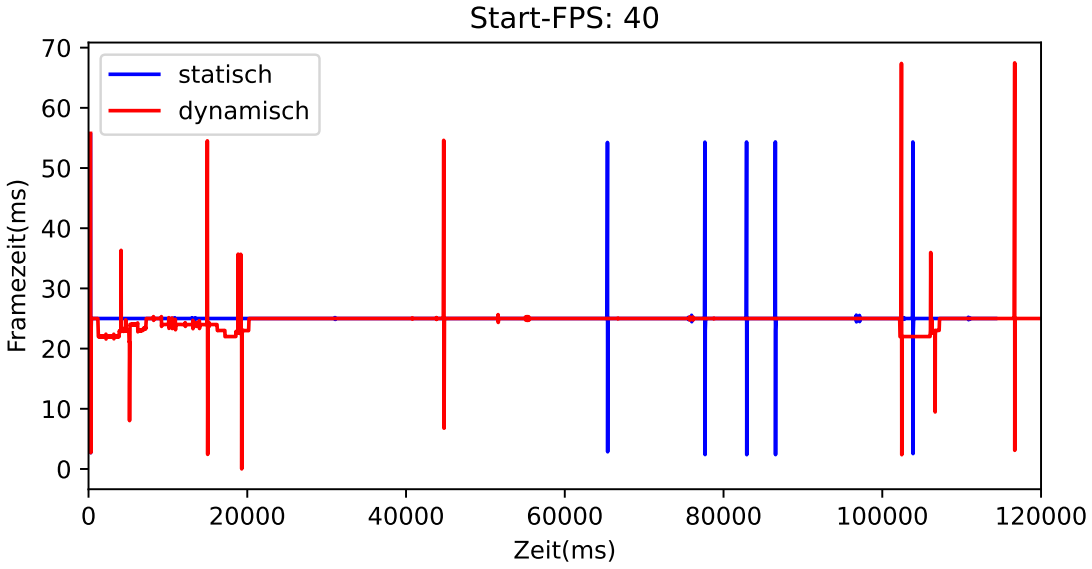


Figure 7.5: Run with 40 FPS

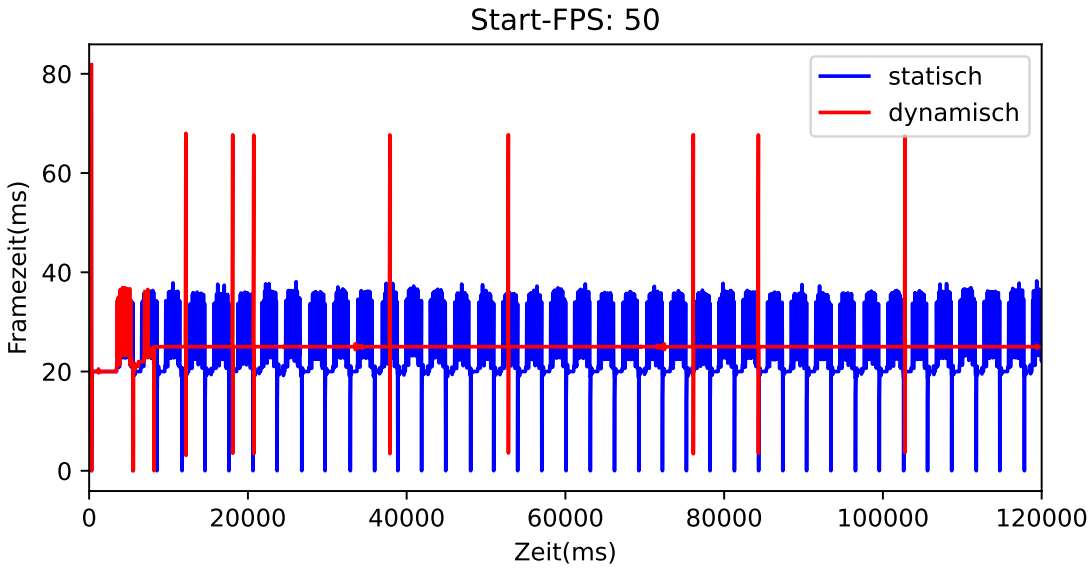


Figure 7.6: Run with 50 FPS

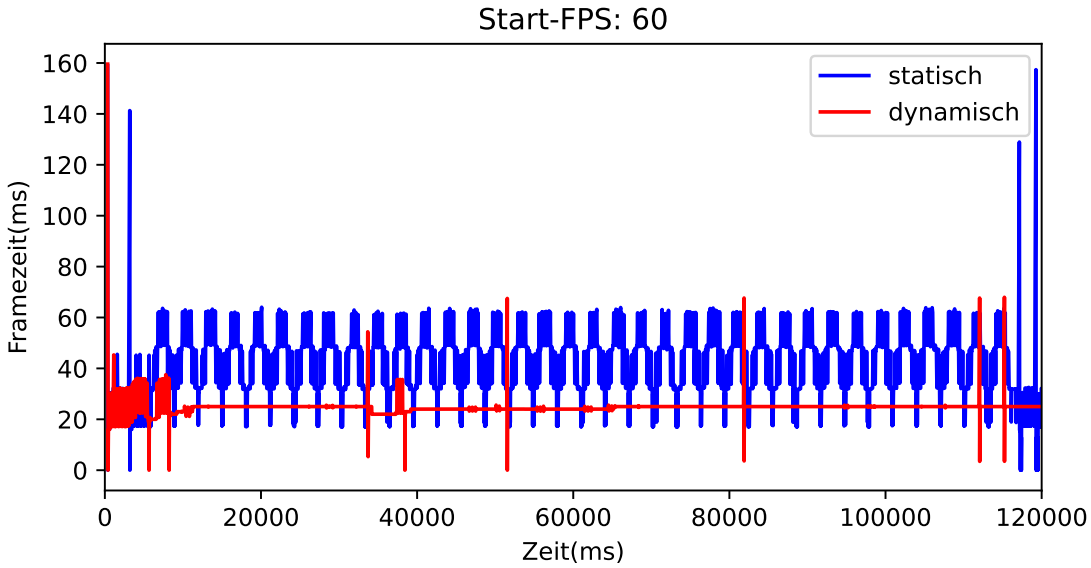


Figure 7.7: Run with 60 FPS

The measurement results were very similar from run to run. For each start FPS value, one measurement is shown graphically below: Measurements were taken for the frame rate values 20 FPS (Fig. 7.3), 30 FPS (Fig. 7.4), 40 FPS (Fig. 7.5), 50 FPS (Fig. 7.6), and 60 FPS (Fig. 7.7). In each figure, the x-axis shows the elapsed measurement time in milliseconds and the y-axis the measured frame times in milliseconds. The blue graph represents the frame times without dynamic frame rate adaptation, and the red graph shows the frame times with dynamic frame rate adaptation. For frame rates up to 40

FPS, it can be seen that the set frame rate is held stable without dynamic frame rate adaptation. From a frame rate of 50 FPS onward, the measurements without dynamic frame rate adaptation show a lot of jitter and many peaks. Across all measurements with dynamic frame rate adaptation, the frame rate can be seen to settle at approx. 40 FPS.

7.3.2 Analysis

The measurement results show that the dynamic frame rate adaptation has a positive influence on the user's gaming experience. This can be seen at two different points. Looking, for example, at the curve for the start FPS value of 20, the static curve shows that this value is held constant. Looking at the curve with dynamic frame rate adaptation, one sees that it first settles and then stays constant at a frame rate of approx. 40 FPS. This shows that the dynamic frame rate adaptation makes the best possible use of the available processing power in order to obtain as smooth an image as possible. The measurement considered here shows that without dynamic frame rate adaptation the user would play at only 20 FPS throughout, which leads to a "choppy" gaming experience. That is, if the start FPS value is set too low, the game engine adapts so that it utilizes the available performance as fully as possible.
Looking in turn at the measurement curve for the start FPS value of 60 FPS, one sees that here the start value was set far too high. Without dynamic frame rate adaptation, peaks in the frame times occur again and again, which is perceived as stuttering and jitter while playing. With dynamic frame rate adaptation, one sees that there are also strong peaks in the frame times at the beginning. This is because the adaptation first has to settle and set the possible frame rate correctly. After the possible frame rate has settled, it can be seen that it is also held constant by the game engine. Settling took approx. 8 to 10 seconds in the measurements. Accordingly, there is no longer any jitter and there are no longer many peaks in the frame times, and the game can again be perceived as smooth.

7.4 Optimal Tile Size

To investigate how the game engine behaves with different tile sizes, the frame times are measured at different tile sizes. In total, five different tile sizes are examined. For each tile size, three runs of two minutes each were measured. Flappy Bird was again played manually, and the dynamic frame rate adaptation was run with the following settings: The start FPS value was 30 FPS (`STARTFPS` = 30.0f), and the statistics task was executed with a period time of 1 second (`STATSPERIODINMS` = 1000). The minimum achievable frame rate during the measurement was 20 FPS (`MIN_FPS` = 20.0f) and the maximum 120 FPS (`MAX_FPS` = 120.0f). The statistics task always used the last 20 measured execution times (`STATSVALUESCOMPARECOUNT` = 20) to calculate the currently possible frame rate. In addition, the buffer share within the frame time was set to 20 %

(UTALIZATION_BUFFER = 0.2f). The frame rate was changed if it already differed from the old one by the value 1 (FPS_THRESHOLD = 1); at most, the current frame rate was changed by the value 5 (FPS_MAX_OFFSET_PER_ITERATION = 5). The statistics task reacted to five deadline misses (DEADLINE_MISS_THRESHOLD = 5) with a frame rate reduction of two FPS (FPS_DECREASE_ON_DEADLINE_MISS = 2). A transfer miss was registered when five transfers (TRANSFER_MISS_THRESHOLD = 5) were still open when the next render job started; the reaction was a frame rate reduction of 2 FPS (FPS_DECREASE_ON_TRANSFER_MISS = 2).

7.4.1 Measurement Results

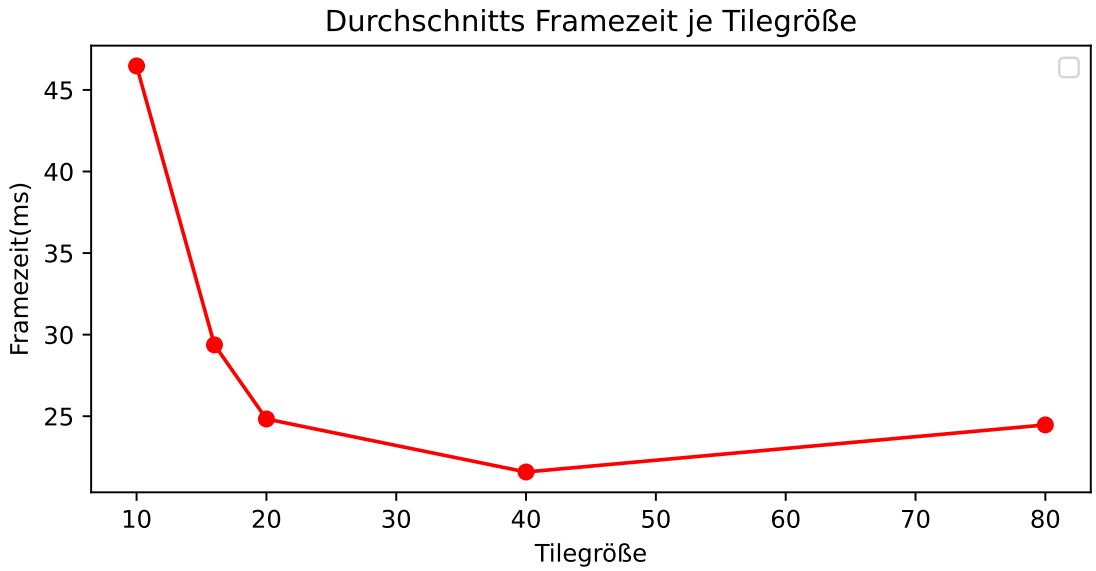


Figure 7.8: Frame times per tile size

Figure 7.8 shows the average frame times per tile size. The x-axis represents the tile sizes, and the y-axis the frame times in milliseconds. It can be seen that there is a minimum of the frame time at a tile size of 40 * 40 pixels. For both larger and smaller tile sizes, the average frame times increase again. For tile sizes smaller than 10 * 10 pixels, no measurement could be carried out because the game engine implementation did not start.

7.4.2 Analysis

The curve of the average times shows advantages and disadvantages of tile rendering. For tile sizes smaller than 20 * 20 pixels, the computation and management overhead outweighs the benefit. This causes the frame times to become larger and only lower frame rates to be achievable. For tile sizes smaller than 10 pixels, the game engine does not start because the memory requirement of the management overhead is too high for the

memory provided by the microcontroller. The larger the tile size becomes, the smaller the frame times become, so that higher frame rates are also achieved. The optimum is at a tile size of 40 * 40 pixels, where a frame time of 21.58 and thus a frame rate of approx. 46 FPS is achieved. For tile sizes that are too large, the frame time increases again. This is because considerably more data has to be sent and the bottleneck caused by the transfer bandwidth of the screen takes effect, since the game engine then detects more transfer misses and accordingly reduces the frame rate.

8 Conclusion

Finally, this section summarizes what was achieved in this bachelor's thesis and gives an outlook on the future.

8.1 What Was Achieved?

In this thesis, a game engine based on concepts from real-time operating systems was successfully developed. With it, games for microcontrollers can now be developed, with the user having to focus only on the design of the game and its logic, without having to implement, for example, the rendering manually. In addition, a dynamic frame rate adaptation with a positive influence on the gaming experience was developed, as well as a physical handheld that can be used as a gaming system for the game engine. The evaluation showed that the game engine is robust but nevertheless entails a certain overhead due to the real-time operating system structures compared with an implementation using a main loop. The requirements that were set could be met in this thesis; however, the testing of different hardware, such as testing different screens with different interfaces, was limited due to time constraints.

8.2 Outlook

The game engine can be extended with many more features. For example, further features can be added at the user layer, such as animations or level systems, in order to be able to develop more extensive games.
In addition, the rendering approach could be extended with raycasting in order to be able to develop 3D games with the game engine. To avoid image stuttering when adapting the frame rate, the frame rate is currently adapted only in steps. Another way to adapt the frame rate as smoothly as possible would be to use a proportional–integral–derivative (PID) controller. This could accordingly also be built into the game engine.

List of Figures

3.1 Components of a game engine 7
3.2 Architecture of the game engine 9
3.3 Architecture of the hardware layer 10
3.4 Architecture of the engine core 11
3.5 Architecture of the user layer 13

4.1 Circuit diagram of the handheld 17
4.2 Layout and 3D view of the PCB 19

5.1 Flow of the logic task 24
5.2 Optimal timing 28
5.3 Flow of the audio task 31
5.4 Period time is too large 31
5.5 Deadline misses 32
5.6 Transfer misses 32

6.1 Excerpt of the game 40

7.1 Run without load 43
7.2 Run with load 43
7.3 Run with 20 FPS 45
7.4 Run with 30 FPS 46
7.5 Run with 40 FPS 46
7.6 Run with 50 FPS 47
7.7 Run with 60 FPS 47
7.8 Frame times per tile size 49

List of Algorithms

5.1 ISR for hardware interrupt processing 22
5.2 Flow of the render task 26
5.3 Flow of the statistics task 33

List of Source Code

5.1 Callback after a transfer 26

Bibliography

[1] Richard Barry and The FreeRTOS Team. *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Release Version 1.1.0. 2023. URL: https://www.freertos.org/Documentation/02-Kernel/07-Books-and-manual/01-RTOS_book (visited on 08/02/2026).

[2] Jonathan Bartlett. "Basic Resistor Circuit Patterns". In: *Electronics for Beginners: A Practical Introduction to Schematics, Circuits, and Microcontrollers*. Springer, 2020, pp. 129–144.

[3] Giorgio Buttazzo. *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. 4th ed. Springer, 2024.

[4] DFRobot. *FireBeetle 2 ESP32-P4 Edge AI Vision Board with MIPI & Wi-Fi 6 | DFRobot*. 2025. URL: https://www.dfrobot.com/product-2915.html (visited on 07/29/2026).

[5] *ESP Timer (High Resolution Timer)*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.2/esp32p4/api-reference/system/esp_timer.html (visited on 07/30/2026).

[6] *ESP-IDF Programming Guide*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.1/esp32p4/index.html.

[7] *ESP32-P4 Series Datasheet Pre-release v0.5*. Espressif Systems. URL: https://dfimg.dfrobot.com/62d52567aa9508d63a4247a1/wikien/30ca0837adaac2d5c26640d7b8e377fe.pdf (visited on 07/12/2026).

[8] Mick Foley. "Advanced Graphics". In: *Tricks of the Mac Game Programming Gurus*. Ed. by Jamie McCornack. Hayden Books, 1995, pp. 121–208.

[9] *FreeRTOS Overview*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.2/esp32p4/api-reference/system/freertos.html (visited on 07/30/2026).

[10] *GPIO & RTC GPIO*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.2/esp32p4/api-reference/peripherals/gpio.html (visited on 07/01/2026).

[11] Jason Gregory. *Game Engine Architecture*. 3rd ed. CRC Press, 2018.

[12] *Implications of Slow or Floating CMOS Inputs*. Texas Instruments. 2021. URL: https://www.ti.com/lit/an/scba004e/scba004e.pdf (visited on 08/02/2026).

[13] *LCD*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.2/esp32p4/api-reference/peripherals/lcd/index.html (visited on 07/12/2026).

[14] *LED Control (LEDC)*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/v6.0.1/esp32p4/api-reference/peripherals/ledc.html (visited on 07/20/2026).

[15] *LED Pattern Game with FreeRTOS on ESP32*. URL: https://github.com/OsaidHamza7/LED-Pattern-Game-with-FreeRTOS-on-ESP32 (visited on 07/31/2026).

[16] Qing Li and Caroline Yao. *Real-Time Concepts for Embedded Systems*. CRC Press, 2003.

[17] Ansgar Meroth and Petre Sora. "Serial Peripheral Interface (SPI)". In: *Sensornetzwerke in Theorie und Praxis: Embedded Systems-Projekte erfolgreich realisieren*. Springer Fachmedien Wiesbaden, 2021, pp. 219–239.

[18] *Miscellaneous System APIs*. Espressif Systems. URL: https://docs.espressif.com/projects/esp-idf/en/latest/esp32p4/api-reference/system/misc_system_api.html (visited on 07/29/2026).

[19] *PTS645 Series 6 mm Tact Switches*. C&K Switches. 2018. URL: https://assets.rs-online.com/v1700169123/Datasheets/faa3b5df6150436956cf2922026c8510.pdf (visited on 06/24/2026).

[20] H. F. Schlaak. "Elektromechanische Relais". In: *Handbuch der Mess-und Automatisierungstechnik im Automobil: Fahrzeugelektronik, Fahrzeugmechatronik*. Springer, 2006, pp. 279–296.

[21] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts*. 10th ed. Wiley, 2018.

[22] *Snake game using an STM32 board and an OLED screen*. URL: https://github.com/rsilverioo/STM32_snake (visited on 07/31/2026).

[23] *The FreeRTOS™ Reference Manual*. Amazon Web Services. URL: https://www.freertos.org/media/2018/FreeRTOS_Reference_Manual_V10.0.0.pdf (visited on 07/07/2026).

[24] *User Manual For 2.8" TFT Touch Shield for Arduino with Resistive Touch Screen (TF028)*. IDUINO. URL: https://asset.conrad.com/media10/add/160267/c1/-/en/001525436ML02/manual-1525436-display-suitable-for-arduino-board-arduino-uno-arduino.pdf (visited on 06/22/2026).

Eidesstattliche Versicherung

(Affidavit)

Adolenko, Maximilian — 252750

Name, Vorname (surname, first name) — Matrikelnummer (student ID number)

■ Bachelorarbeit (Bachelor's thesis) ☐ Masterarbeit (Master's thesis)

Titel (Title)

Realisierung einer Game-Engine in eingebetteten Echtzeitbetriebssystemen

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem oben genannten Titel selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

I declare in lieu of oath that I have completed the present thesis with the above-mentioned title independently and without any unauthorized assistance. I have not used any other sources or aids than the ones listed and have documented quotations and paraphrases as such. The thesis in its current or similar version has not been submitted to an auditing institution before.

Dortmund, 07.08.2026 — Adolenko

Ort, Datum (place, date) — Unterschrift (signature)

Belehrung:
Wer vorsätzlich gegen eine die Täuschung über Prüfungsleistungen betreffende Regelung einer Hochschulprüfungsordnung verstößt, handelt ordnungswidrig. Die Ordnungswidrigkeit kann mit einer Geldbuße von bis zu 50.000,00 € geahndet werden. Zuständige Verwaltungsbehörde für die Verfolgung und Ahndung von Ordnungswidrigkeiten ist der Kanzler/die Kanzlerin der Technischen Universität Dortmund. Im Falle eines mehrfachen oder sonstigen schwerwiegenden Täuschungsversuches kann der Prüfling zudem exmatrikuliert werden. (§ 63 Abs. 5 Hochschulgesetz - HG -).

Die Abgabe einer falschen Versicherung an Eides statt wird mit Freiheitsstrafe bis zu 3 Jahren oder mit Geldstrafe bestraft.

Die Technische Universität Dortmund wird ggf. elektronische Vergleichswerkzeuge (wie z.B. die Software „turnitin") zur Überprüfung von Ordnungswidrigkeiten in Prüfungsverfahren nutzen.

Die oben stehende Belehrung habe ich zur Kenntnis genommen:

Official notification:
Any person who intentionally breaches any regulation of university examination regulations relating to deception in examination performance is acting improperly. This offense can be punished with a fine of up to EUR 50,000.00. The competent administrative authority for the pursuit and prosecution of offenses of this type is the Chancellor of TU Dortmund University. In the case of multiple or other serious attempts at deception, the examinee can also be unenrolled, Section 63 (5) North Rhine-Westphalia Higher Education Act (*Hochschulgesetz, HG*).

The submission of a false affidavit will be punished with a prison sentence of up to three years or a fine.

As may be necessary, TU Dortmund University will make use of electronic plagiarism-prevention tools (e.g. the "turnitin" service) in order to monitor violations during the examination procedures.

I have taken note of the above official notification:*

Dortmund, 07.08.2026 — Adolenko

Ort, Datum (place, date) — Unterschrift (signature)

*Please be aware that solely the German version of the affidavit ("Eidesstattliche Versicherung") for the Bachelor's/ Master's thesis is the official and legally binding version.